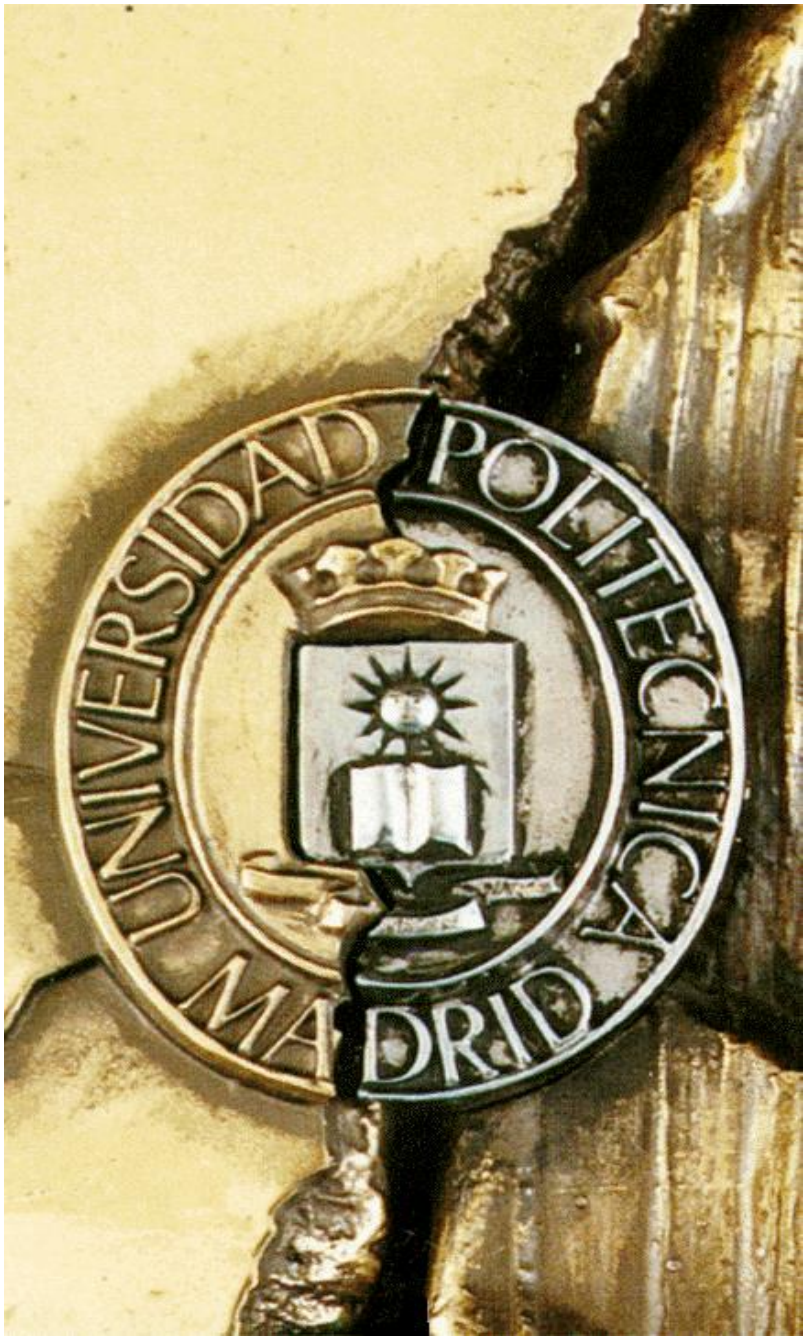


SISTEMAS OPERATIVOS

SISTEMA DE FICHEROS

Víctor Robles Forcada
Mª de los Santos Pérez Hernández

Curso 2024/25

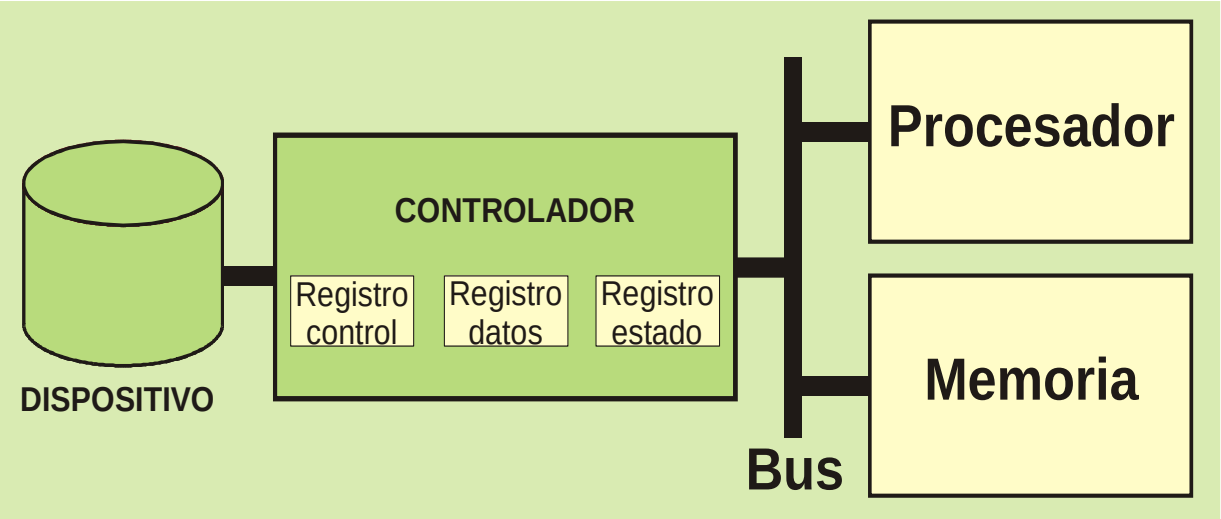


POLITÉCNICA

CONCEPTOS BÁSICOS DE E/S

HARDWARE DE E/S

© UPM 2022



- Arquitectura con mapa de E/S propio IN y OUT (p.e. Intel)
- Arquitectura con mapa de E/S en memoria LOAD y STORE (p.e. ARM)

El acceso a los dispositivos es

- Complejo
  - Detalles físicos de los dispositivos
  - Dependiente de las direcciones físicas
- Sin protección
  - Si el usuario accede a nivel físico no tiene restricciones. El controlador del dispositivo no limita

datsi © UPM 2022

Hardware de E/S

- **Dispositivos de E/S**
  - **De bloques (discos, cintas)**
  - **De caracteres (teclado, ratón)**
  - **Acceso aleatorio**
  - **Acceso secuencial**
- **Controladores de dispositivos**
  - **E/S programada: No concurrencia E/S-procesador**
  - **Interrupciones: Concurrencia E/S-procesador**
  - **DMA: Máxima concurrencia E/S-procesador**

Objetivos del SO en E/S

1. **Controlar el funcionamiento de los dispositivos de E/S**
2. **Facilitar el manejo de los dispositivos de E/S a través de interfaces**
3. **Proporcionar mecanismos de protección**
4. **Explotar la concurrencia E/S-procesador**



POLITÉCNICA

SISTEMA DE ALMACENAMIENTO

DISCOS, PARTICIONES Y VOLÚMENES

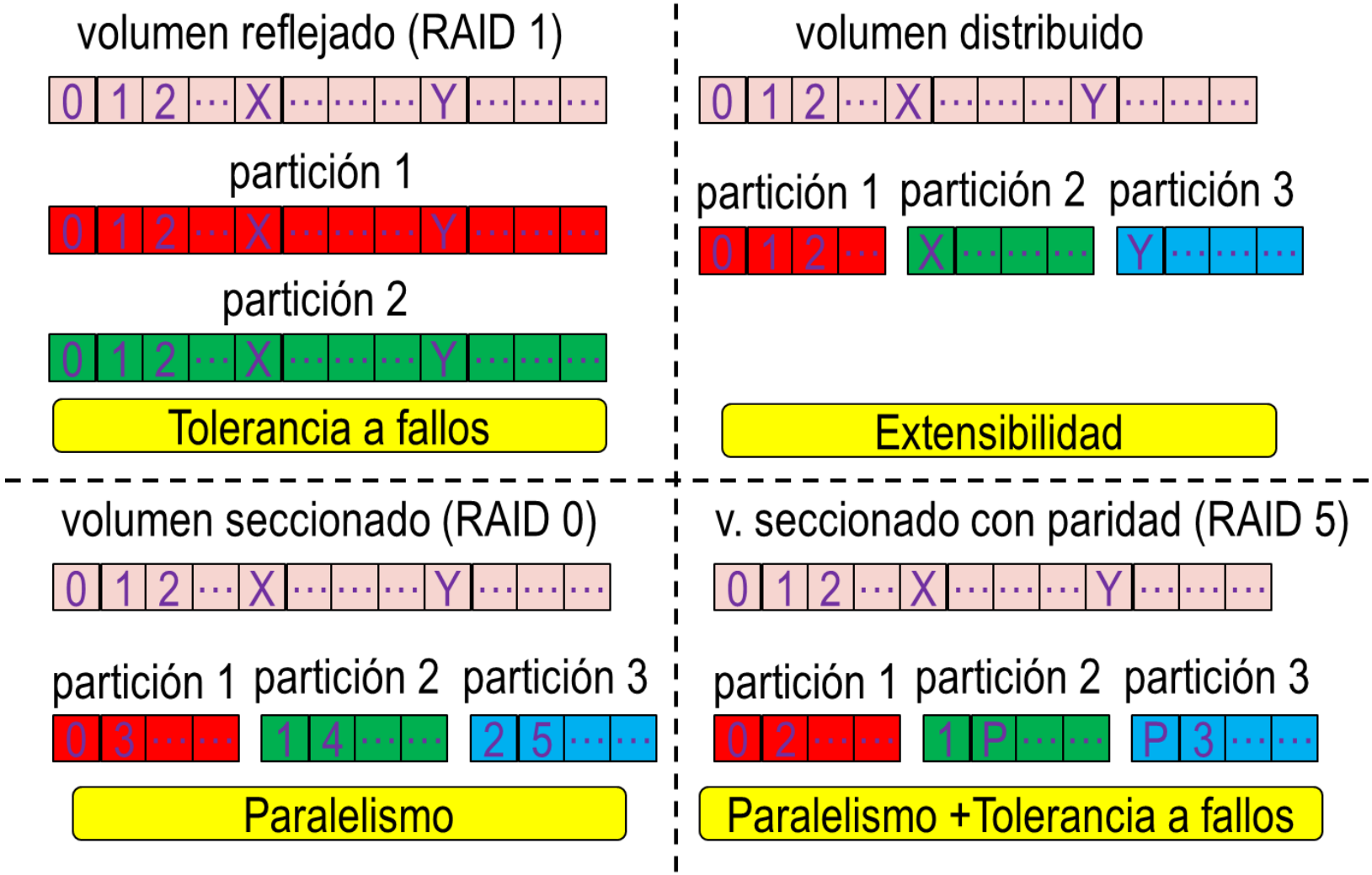
datsi © UPM 2022

POLITÉCNICA

- **Gran variedad de tipos de discos (estudiados en estructura de computadores)**
  - **Tecnología de almacenamiento: magnéticos (HDD), de estado sólido (SSD)…**
  - **Interfaz de conexión: PATA, SATA, NVME, SCSI, SAS...**
- **Partición: porción lógica de un disco a la que se la dota de identidad propia**
  - **Puede ser manipulada por SO como una entidad lógica independiente**
  - **Tabla de particiones indica el principio, el tamaño y el tipo de cada partición**
  - **Fiabilidad (fallo en partición solo afecta a esta) y flexibilidad (p.e. solo lectura)**
  - **En PC: antes BIOS/MBR; actualmente UEFI/GPT**
    - Mejora fiabilidad y límite en número de particiones y su tamaño
- **Volumen: espacio de almacenamiento lógico basado en varias particiones**
  - **Distintos esquemas de composición que pueden ofrecer:**
    - Fiabilidad: duplica información entre particiones o uso de paridad
    - Capacidad de crecimiento del volumen añadiendo nuevas particiones
    - Eficiencia permitiendo acceder a fichero en paralelo en varios discos
- **SO puede usar disco, partición o volumen como soporte de un s. de ficheros**
- **Volumen: fiabilidad, flexibilidad y/o eficiencia, pero con mayor sobrecarga**

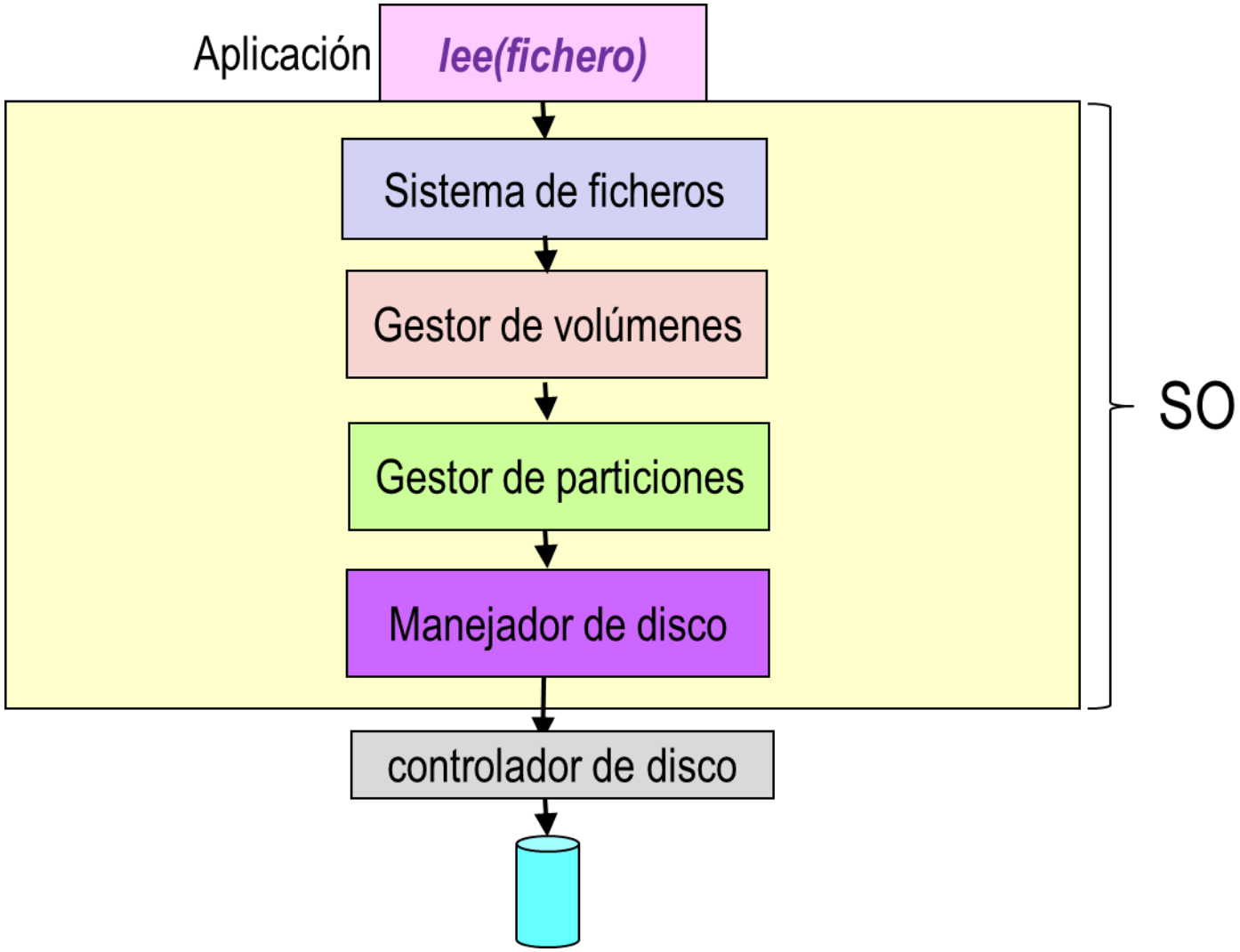
datsi © UPM 2022

POLITÉCNICA



JERARQUÍA COMPLETA DE ALMACENAMIENTO

datsi © UPM 2022





POLITÉCNICA

CONCEPTO DE SISTEMA DE FICHEROS

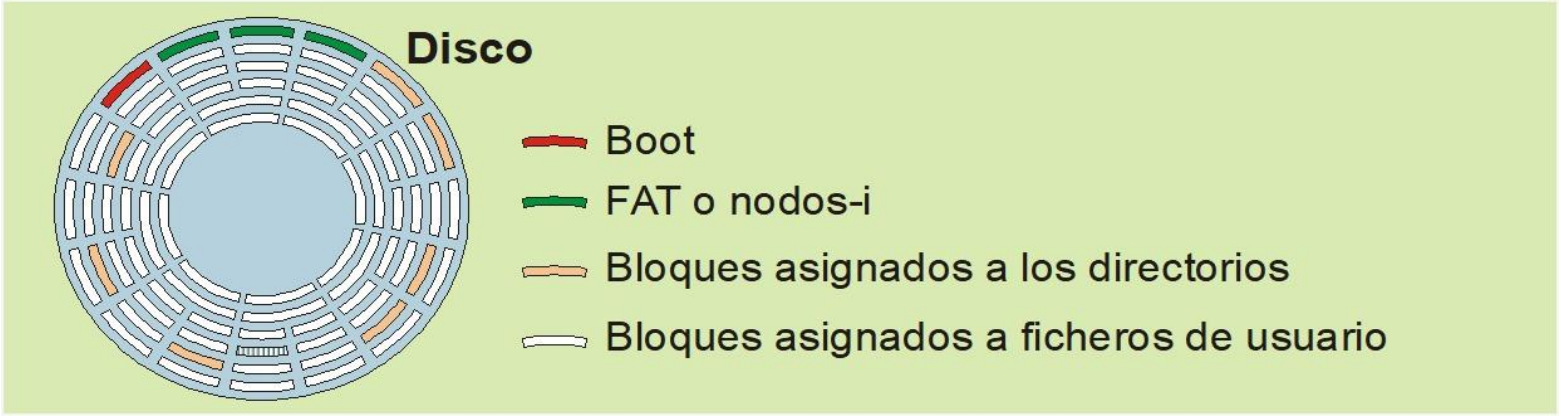
datsi © UPM 2022

Fichero. Conjunto de informaciones relacionadas que se almacenan en almacenamiento secundario y que se identifica mediante un nombre.

Sistema de ficheros. Conjunto autónomo de informaciones incluidas en una unidad de almacenamiento (partición o volumen) que permiten su explotación. Se compone de:

- **Información neta: Ficheros de usuario (programas y datos).**
- **Metainformación**
  - **Estructura física de los ficheros – descripción física de los ficheros DFF**
    - Información asociada a los ficheros.
  - **Directorios.**





SISTEMA DE FICHEROS FAT

datsi © UPM 2022



Utilizado por defecto en MS-DOS y Microsoft Windows 95|98

– Windows usa actualmente NTFS (aunque requiere una partición FAT para arranque)

– Uso en dispositivos de almacenamiento extraíbles (discos USB, tarjetas SD…)

Espacio dividido en

– BOOT: Bloque de carga: Arranque del sistema operativo

– FAT: Tabla de asignación de ficheros

– Copia de FAT: Copia de seguridad

– Directorio Raíz: Directorio principal del volumen

– Zona de directorios y ficheros

Sistemas de Ficheros actuales
FAT32: Partición máxima 4TB, fichero máximo fichero 2^32 bytes (4GiB)
exFAT: Tamaño máximo de fichero 2^64 bytes (16 Exbibytes)

datsi © UPM 2022

POLITÉCNICA



Estructura:

– BOOT. Bloque de carga: Arranque del sistema operativo

– Superbloque: Información descriptiva de la estructura del sistema de archivos

– Mapa de bits de i-nodos: Un bit para cada i-nodo de la sección de i-nodos indicando libre/ocupado

– Mapa de bits de agrupaciones: Un bit para cada agrupación indicando libre/ocupado

– i-nodos: Tantos i-nodos como número de ficheros (y directorios) que puede albergar el sistema

– Agrupaciones de datos para almacenar los ficheros y directorios

Sistemas de Ficheros actuales

UFS2: Unix File System

EXT4: Fourth Extended Filesystem. Volúmenes con tamaños de hasta 1 exbibyte (EiB) y archivos individuales con tamaños de hasta 16 tebibytes (TiB) con el tamaño de bloque estándar de 4 KiB

datsi © UPM 2022

POLITÉCNICA

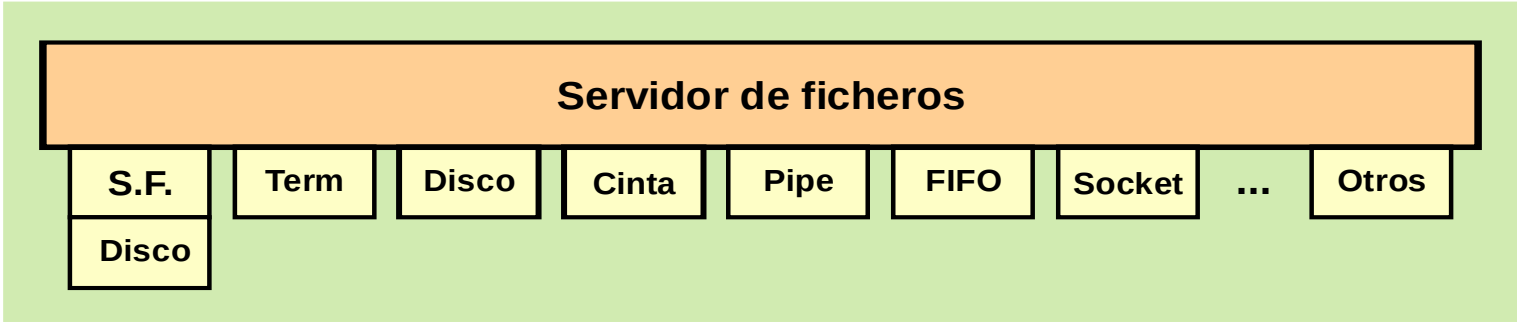
Servidor de Ficheros

Capa de software entre dispositivos y usuarios que

- **Suministra una visión lógica uniforme de los dispositivos, presentándolos como ficheros**
- **Ofrece primitivas de acceso cómodas e independientes de los detalles físicos**
- **Incorpora mecanismos de protección**

El servidor de ficheros permite manejar los siguientes tipos de objetos:

- **Ficheros de usuario (datos y programas)**
- **Ficheros especiales : representa dispositivo orientado a carácter (ej. terminal)**
  representa dispositivo orientado a bloque (ej. disco)
- **Mecanismos de comunicación con y sin nombre**
  - **Pipe, FIFO, socket UNIX**



Unidades básicas de información en bytes

POLITÉCNICA

Nombre de magnitud	Símbolo	Factor en sistema decimal	Valor en sistema binario (en Bytes)
Byte	B	10^0	1
Kilobyte	KB	10^3	1.000
Megabyte	MB	10^6	1.000.000
Gigabyte	GB	10^9	1.000.000.000
Terabyte	TB	10^{12}	1.000.000.000.000
Petabyte	PB	10^{15}	1.000.000.000.000.000
Exabyte	EB	10^{18}	1.000.000.000.000.000.000
Zettabyte	ZB	10^{21}	1.000.000.000.000.000.000.000
Yottabyte	YB	10^{24}	1.000.000.000.000.000.000.000.000

Nombre de magnitud	Símbolo	Factor en sistema binario	Valor en sistema binario (en Bytes)
Byte	B	2^0	1
Kibibyte	KiB	2^{10}	1.024
Mebibyte	MiB	2^{20}	1.048.576
Gibibyte	GiB	2^{30}	1.073.741.824
Tebibyte	TiB	2^{40}	1.099 511.627.776
Pebibyte	PiB	2^{50}	1.125 899.906.842.624
Exbibyte	EiB	2^{60}	1.152 921.504.606.846.976
Zebibyte	ZiB	2^{70}	1.180 591.620.717.411.303.424
Yobibyte	YiB	2^{80}	1.208 925.819.614.629.174.706.176



POLITÉCNICA

FICHEROS

VISIÓN FÍSICA DE UN FICHERO

datsi © UPM 2022

Sector: Unidad mínima de transferencia que puede manejar el controlador de disco (2^m bytes, normalmente 2^9)

Bloque: Es un conjunto de sectores de disco y es la unidad de transferencia mínima que usa el sistema de ficheros (bloque = 2^n sectores)

- **El bloque se puede direccionar de manera independiente**

Agrupación: Conjunto de bloques que se utilizan como una unidad lógica de gestión de almacenamiento (agrupación = 2^p bloques).

- **Reserva espacio**

- **Única para cada sistema de ficheros y definible por el usuario**

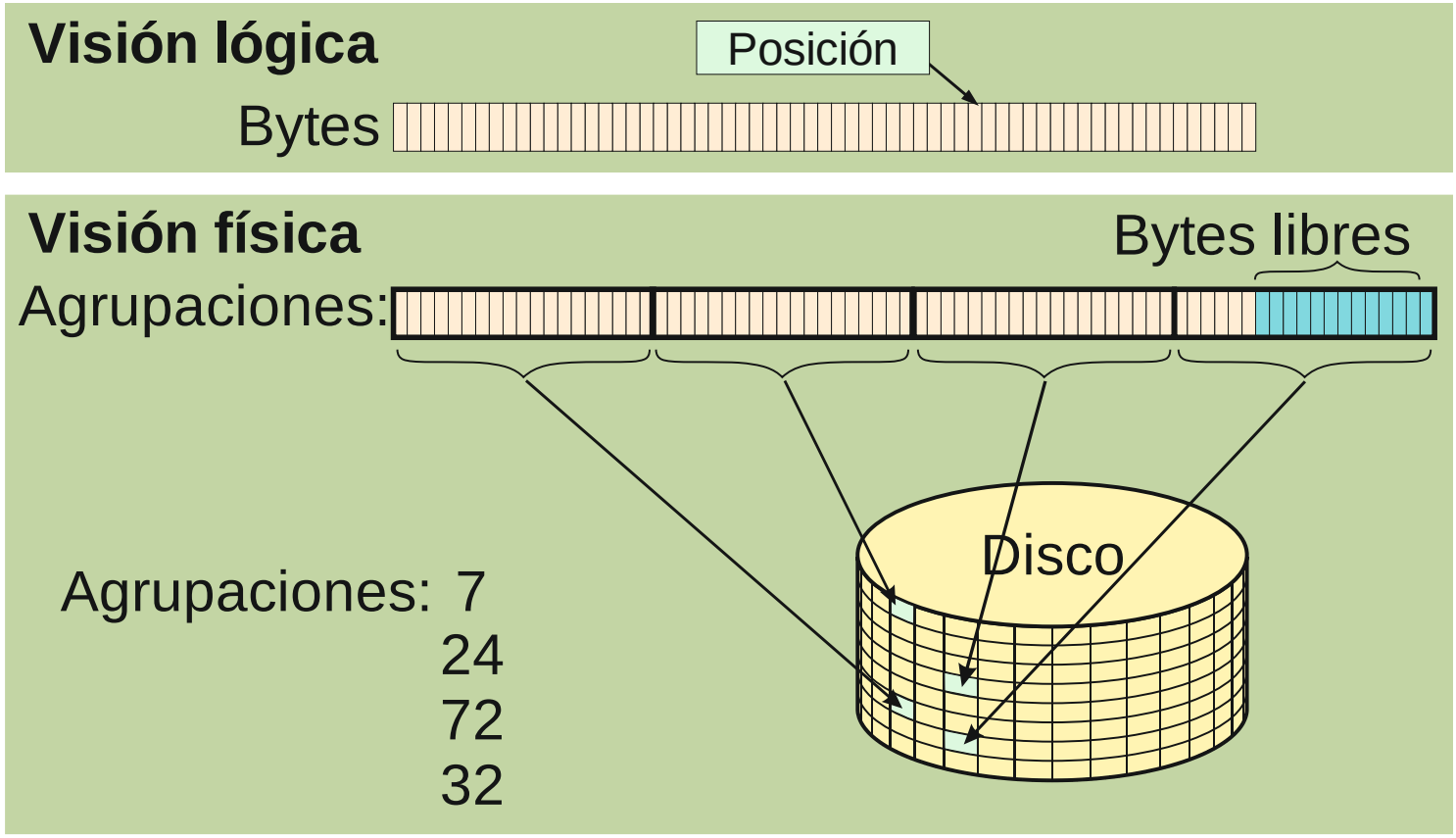
Al fichero se le asignan agrupaciones, pero se accede siempre en bloques

En algunos sistemas Bloque = Agrupación

ESTRUCTURA LÓGICA Y FÍSICA DE UN FICHERO

datsi © UPM 2022

- **Estructura lógica del fichero. Secuencia ordenada de bytes con un puntero que indica la posición a partir de la cual se lee o escribe.**
  - **Sobre esta cadena de bytes se pueden montar otras estructuras: por ejemplo, conjunto de registros.**
- **Estructura física del fichero. Secuencia ordenada de agrupaciones**



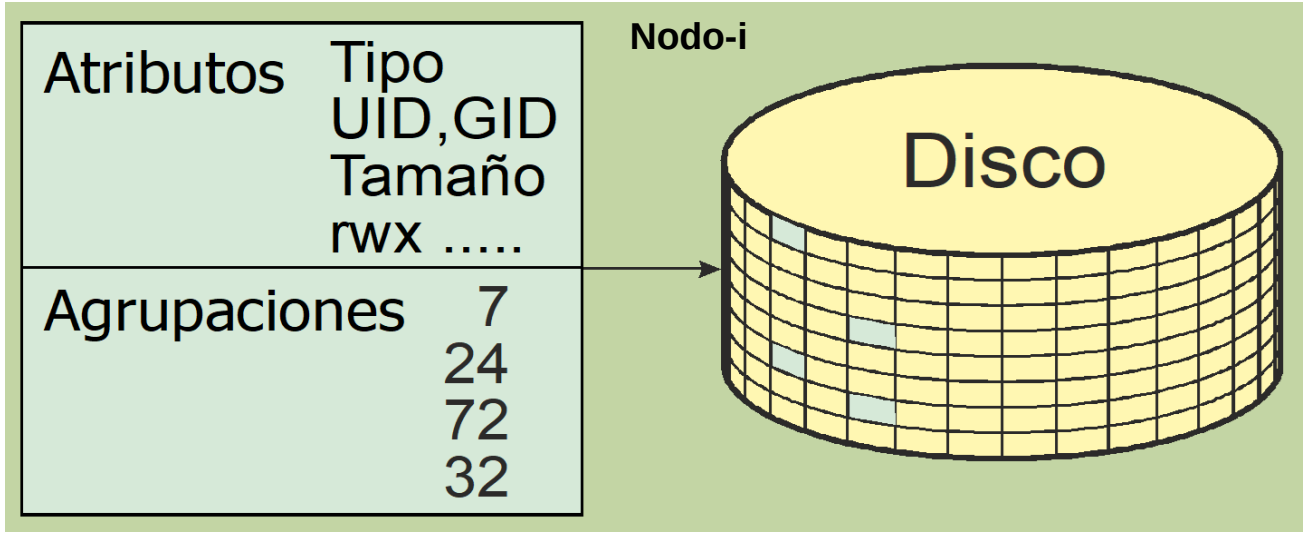
IDENTIFICADOR DE DESCRIPCIÓN FÍSICA DE FICHERO (IDFF)

datsi © UPM 2022

DFF: Estructura de información que recoge la descripción física del fichero

IDDF: Número identificador del DFF.

- **ATRIBUTOS + LOCALIZACIÓN (secuencia ordenada de Agrupaciones)**
- **Almacenada en el propio disco**
- **Diferentes denominaciones**
  - **Entrada de directorio de SF de tipo FAT en MS-DOS**
  - **Registro MFT (Master File Table) de NTFS (New Technology File System) Microsoft**
  - **i-nodo en UNIX**



CARACTERÍSTICAS DE LOS FICHEROS

datsi © UPM 2022

POLITÉCNICA

Atributos de un fichero en UNIX

- Identificador único del fichero (número del nodo-i)
- Tipo de fichero (regular, directorio, dispositivo de bloques, …)
- Dueño y grupo (uid y gid del propietario o dueño)
- Información de protección (bits de protección rwx y otros)
- Tamaño real en bytes
- Hora y fecha de creación
- Hora y fecha del último acceso
- Hora y fecha de la última modificación
- Número de enlaces (número de nombres)

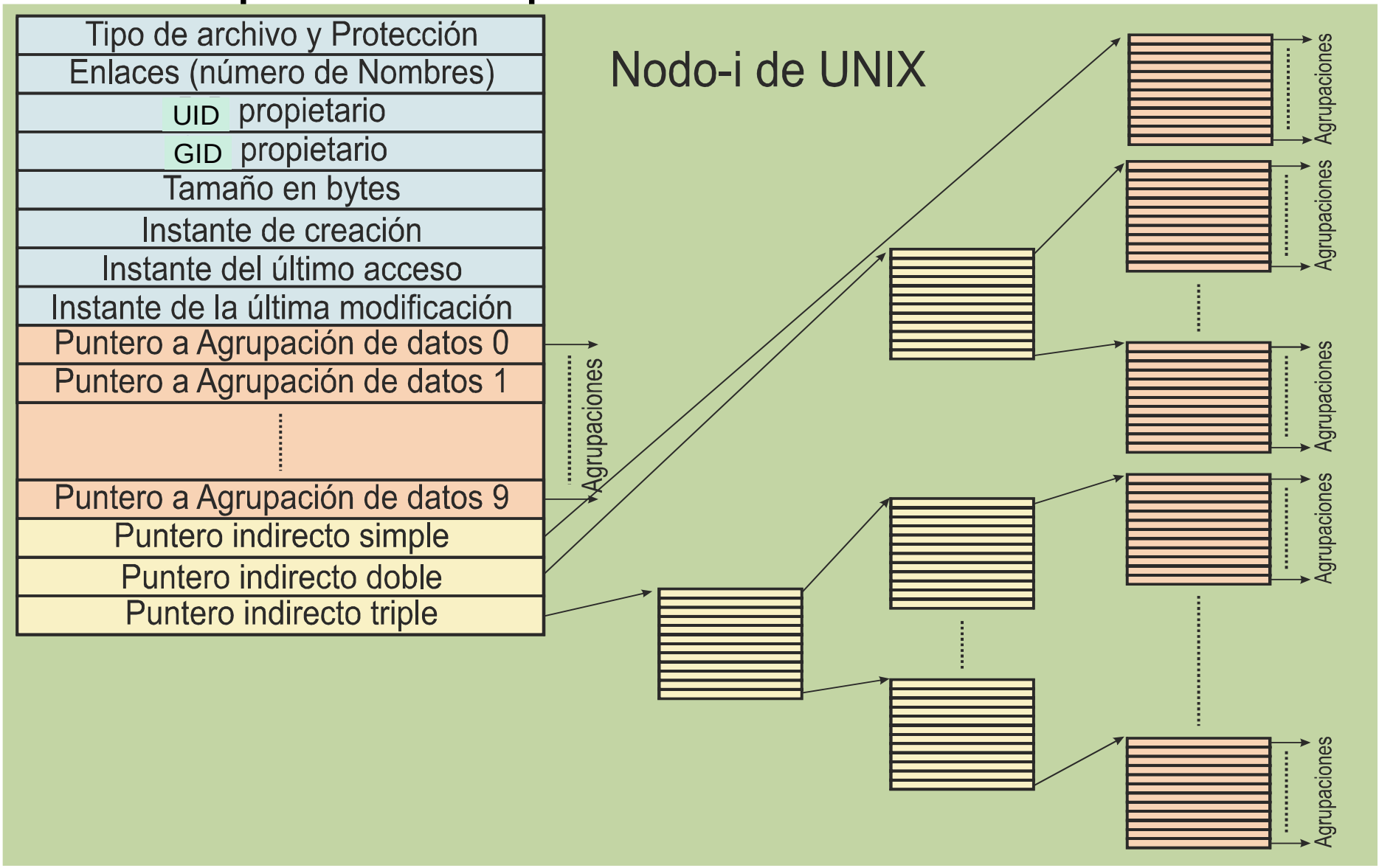
Tamaño máximo de un fichero

- Depende de las limitaciones establecidas por:
  - La metainformación (Atributo tamaño real, direccionamiento de las agrupaciones –tamaño direcciones y nº de direcciones–)
  - Las agrupaciones de datos disponibles (agrupaciones existentes en disco o cuotas establecidas por el administrador)

datsi © UPM 2022

POLITÉCNICA

- **Tamaño típico 128 B con punteros de 4B.**



Cálculos sobre un Sistema de Ficheros

datsi © UPM 2022



1. **Tamaño máximo del disco dados el tamaño de la agrupación y la longitud de la dirección de agrupación**
   - $2^{tamDireccion}$ * tamAgrup
2. **Dado el tamaño de un disco calcular el tamaño del mapa de bits de agrupaciones**
   numAgrupMapa_agrup = [tamDisco / tamAgrup / 8 / tamAgrup]
3. **Dado el número de ficheros que puede tener un SF calcular el tamaño del mapa de bits de i-nodo**
   - numAgrupMapa_inodos = [num_fich / tamAgrup / 8]
4. **Dado el número de ficheros que puede tener un SF calcular el tamaño de la zona de i-nodos**
   - numAgrupZona_inodos = [(num_fich * tamInodo) / tamAgrup]
5. **Máximo número de ficheros/directorios, dado el número de agrupaciones del mapa de bits de i-nodos**
   - num_fich = numAgrupMapa-inodos * tamAgrup * 8
6. **Máximo número de agrupaciones y máx tamaño de disco, dado el número de agrupaciones del mapa de bits de agrupaciones**
   - Max_agrup = numAgrupMapa-agrup * tamAgrup * 8
   - Max_tam_disco = Max_agrup * tamAgrup
7. **Espacio (en Bytes) ocupado por los datos de un fichero**
   - [(tamFich/tamAgrup)] * tamAgrup
8. **Máximo tamaño de un fichero**
   - Restringido por el tamaño de disco
   - Restringido por la longitud del atributo “tamaño del fichero” del i-nodo
     - 2^longAtrTamFic - 1
   - Restringido por lo máximo direccionable desde los punteros
     Num_directos * tamAgrup +
     Num_ind_simple * (tamAgrup/tamDireccion) * tamAgrup +
     Num_ind_doble * $(tamAgrup/tamDireccion)^2$ * tamAgrup +
     Num_ind_triple * $(tamAgrup/tamDireccion)^3$ * tamAgrup

datsi © UPM 2022

Asignación de espacio al fichero

- Se hace por agrupaciones.
- El fichero puede quedar disperso o fragmentado. Sus agrupaciones no tienen por qué ser contiguas.
- Objetivo de la desfragmentación: mejorar el acceso secuencial.
- SSD no se desfragmentan.

Gestión del espacio libre

- Mapas de bits de agrupaciones y de i-nodos
  - Agrupaciones libres/ocupadas.
  - i-nodos libres/ocupados.

datsi © UPM 2022

Vida del fichero

- **Se crea.**
  - **Se abre → se genera un fd (descriptor de fichero).**
    - Se escribe y lee (el fichero puede crecer).
  - **Se cierra.**
- **Se borra.**

- **Con los siguientes mandatos indicar qué cambios se producen en el Sistema de Ficheros**
  - ***cp fich1.txt fich2.txt***
  - ***mv fich1.txt fich2.txt***

Ejercicio 4 (4 puntos) Para un Sistema de Ficheros tipo UNIX sobre un disco de 2 TiB, con direcciones de 64 bits, sectores de 512 B, bloques de 2 KiB y agrupaciones de 2 bloques, explique:

a) ¿Cuánto espacio de disco ocuparían los datos de un fichero de 1 kilo byte?
b) ¿Cuántas direcciones de agrupación caben en una agrupación?
c) ¿Cuántos accesos a disco se necesitan para leer el byte 409600000 de un archivo dado?
d) Exprese la capacidad de direccionamiento del inodo de este Sistema de Ficheros en bytes.



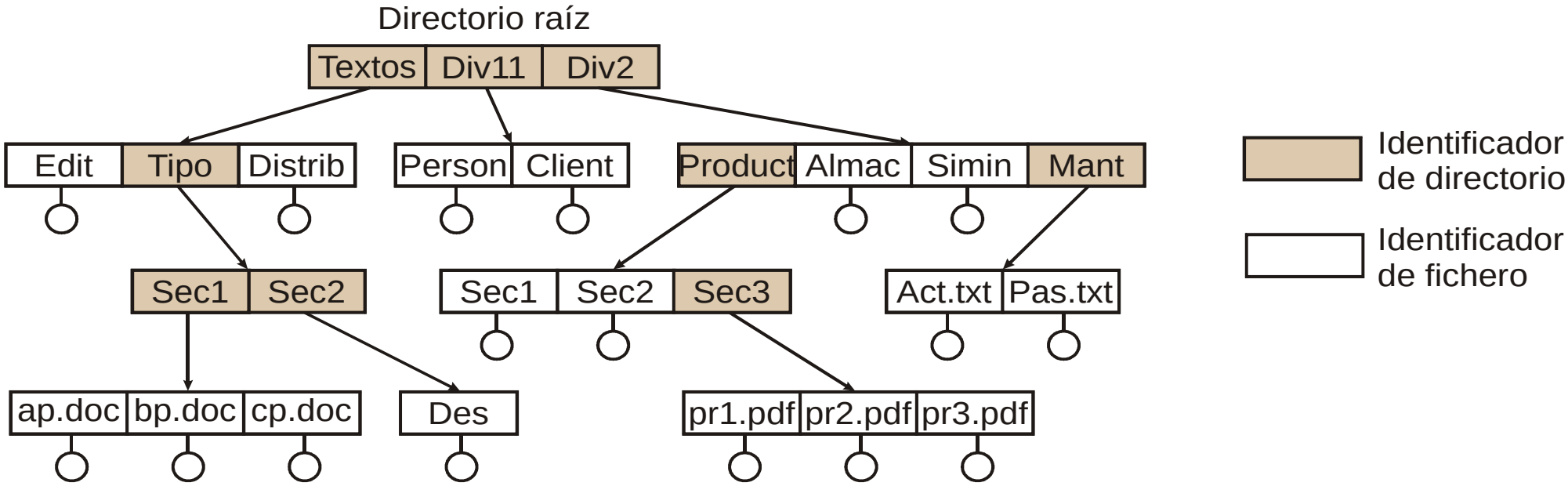
POLITÉCNICA

DIRECTORIOS

datsi © UPM 2022

Estructura lógica: Esquema jerárquico de nombres:

- **Directorio raíz.**
- **Subdirectorios.**



- **UNIX: `/usr/include/stdio.h`**
- **Windows: `C:\DocenciaSO\sos2\transparencias\pipes.ppt`**

Directorio único versus directorio por dispositivo. Operación de montado

ESTRUCTURA LÓGICA DE LOS DIRECTORIOS

- **Un directorio es una tabla con entradas que asocian nombres con nodos_i**

Nombre	nº nodo_i
.	146
..	43
Sec1	239
Sec2	240

- **Esquema jerárquico. La organización jerárquica de un directorio:**
  - **Simplifica el nombrado de ficheros (nombres únicos)**
  - **Proporciona una gestión de la distribución => agrupar ficheros de forma lógica (mismo usuario, misma aplicación)**
- **Cuando se abre un fichero el SO busca el nombre en la estructura de directorios y comprueba los privilegios. Es una operación larga y costosa**
- **Atributo Número de enlaces: Indica el número de nombres de un determinado fichero o directorio**

DIRECTORIO DE TRABAJO O ACTUAL

datsi © UPM 2022

- El SO mantiene en el BCP (Block Control Process) el nombre del directorio actual o de trabajo
- El SO mantiene en memoria el nodo_i del directorio de trabajo
- Un proceso puede cambiar su directorio de trabajo
  - El SO comprueba los permisos antes del cambio
- Nombre absoluto
  - Es el nombre desde el directorio raíz
  - Empieza por / en UNIX o por la unidad en Windows
- Nombre relativo
  - Es el nombre a partir del directorio de trabajo
  - Ahorro de accesos a disco
    - Se parte del nodo_i del directorio de trabajo
    - Se requieren menos accesos a disco para análizar el nombre relativo que el nombre absoluto

NOMBRADO DE FICHEROS Y DIRECTORIOS

datsi © UPM 2022

Nombres de fichero y directorio UNIX

- Nombre completo (empieza por /)

  `/usr/include/stdio.h`

- Nombre relativo al directorio actual o de trabajo (no empieza por /)

  `stdio.h` asumiendo que `/usr/include` es el directorio actual.

- La entradas `.` (propio directorio) y `..` (directorio padre) pueden utilizarse para formar rutas de acceso

  `../include/stdio.h`

  `./../include/stdio.h`

  `/usr/./include/../include/stdio.h`

  Estos tres ficheros hacen referencia a `stdio.h` si `/usr/include` es el directorio actual

- Árbol único (los dispositivos se montan en un punto del árbol)

Nombres de fichero y directorio Windows

- En lugar de utilizar /, se utiliza \
- Entradas . (propio directorio) y .. (directorio padre)
- Varios árboles (uno por sistema de ficheros). NTFS permite montado

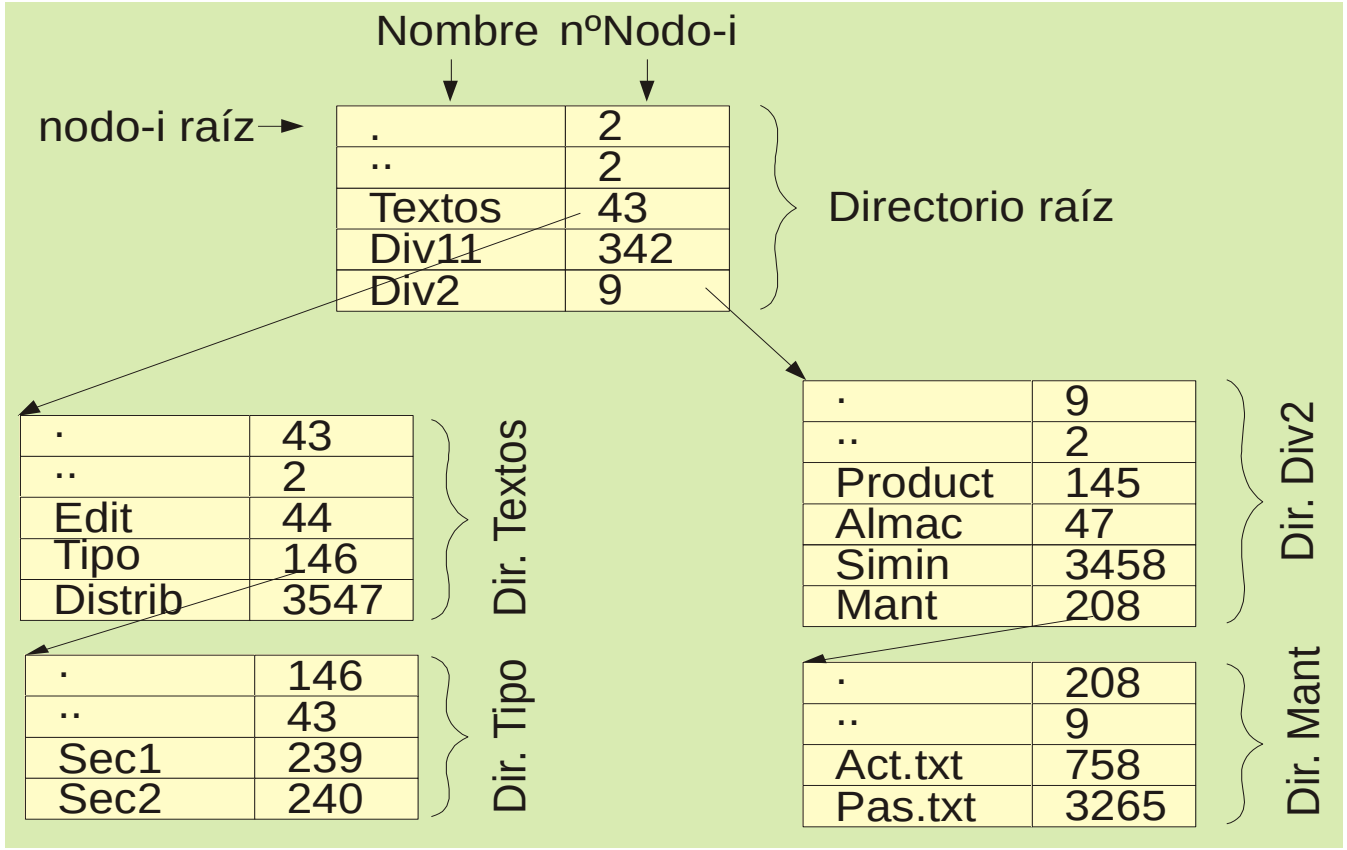
ESTRUCTURA FÍSICA DE LOS DIRECTORIOS

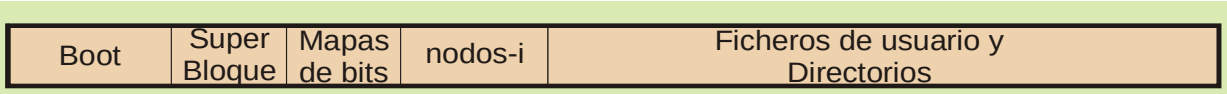
datsi © UPM 2022

Estructura física: Tabla Nombre-IDFF. (En UNIX es tabla Nombre-nº i-nodo)

IDFF: identificador de descripción física de fichero. Permite obtener la tabla que describe el fichero (en UNIX es el i-nodo).

Para alcanzar el IDFF (n.º i-nodo) de un fichero hay que recorrer el árbol.





- **Con los siguientes mandatos indicar qué cambios se producen en el Sistema de Ficheros**
  - ***cp fich1.txt fich2.txt***
  - ***mv fich1.txt fich2.txt***

Sea un sistema de ficheros UNIX con las siguientes características:

- Tamaño de bloque: 1024 bytes.
- Tamaño de agrupación: 1 bloque.
- Campos del nodo-i: Atributos (permisos), tipo de elemento: fichero regular (fich) o directorio (dir), usuario, grupo, número de enlaces, localización (agrupaciones)

Además, se tiene la siguiente visión parcial del contenido del sistema de ficheros:

nodo-i	permisos	tipo	usuario	grupo	enlaces	Localización
2	rwx r-x r-x	dir	root	root	**xxx**	100
3	rwx r-x r-x	dir	root	root	**yyy**	101
4	rwx rwx ---	dir	juan	graf	2	102
5	rwx r-x ---	dir	blanca	graf	2	103
6	rwx r-x ---	dir	irene	prog	2	104
7	rwx r-s ---	fich	juan	graf	1	107,108
8	rw- rw- rw-	fich	juan	graf	1	106
9	rw- --- ---	fich	blanca	graf	**zzz**	105

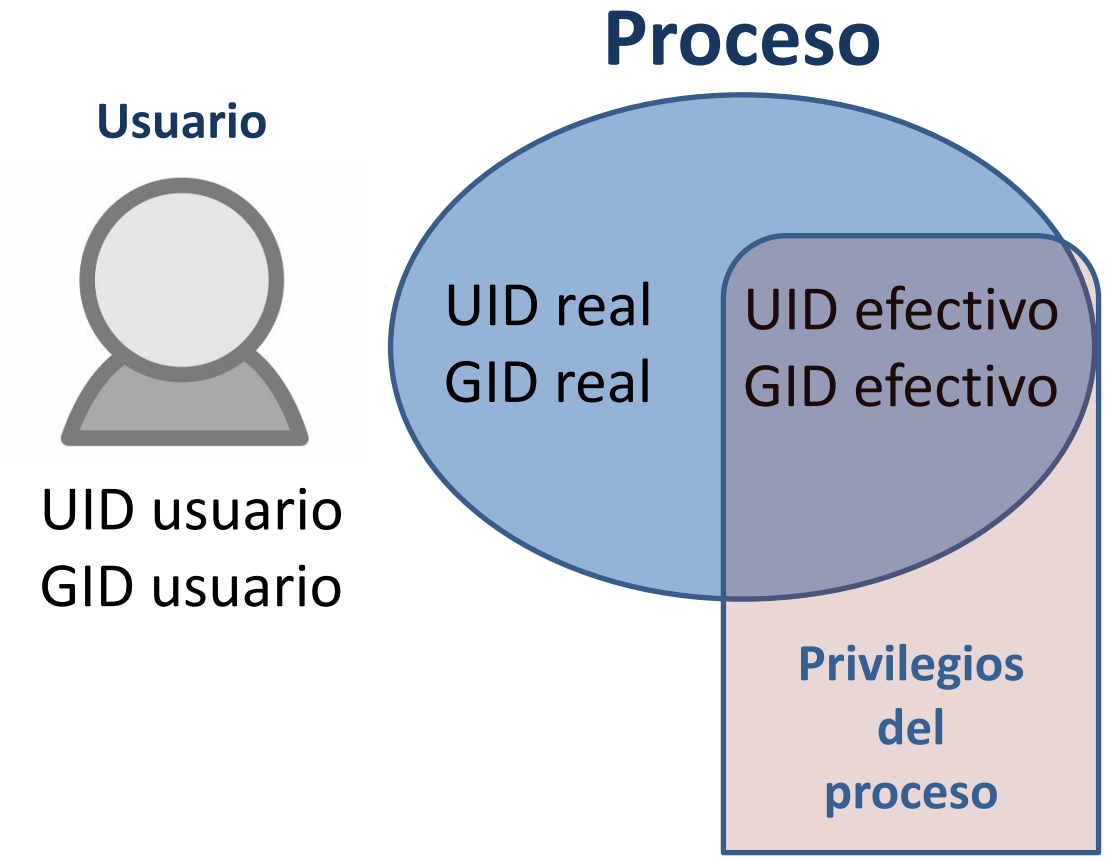
Agrup 100		Agrup 101		Agrup 102		Agrup 103		Agrup 104		Agrup 105	Agrup 106
.	2	.	3	.	4	.	5	.	6	1234567890	123abc456def
..	2	..	2	..	3	..	3	..	3		
home	3	juan	4	ejecutor	7	fich2.txt	9	dat.txt	21		
tmp	15	blanca	5	fich.txt	8	new	12	real	18		
bin	37	irene	6			increm.dat	19	fich2.txt	9		

Responda a las siguientes preguntas

a) (i) Indica los valores **xxx** **yyy** y **zzz** de la primera tabla del sistema de ficheros.
(ii) Pinta la estructura del árbol de ficheros/directorios resultante.

PROTECCIÓN

IDENTIFICADORES DE SEGURIDAD DEL PROCESO



Fichero

i-nodo del fichero

- Información de protección [rwx rwx rwx]
- UID propietario
- GID propietario

En general

UID real = UID usuario	UID efectivo = UID real
GID real = GID usuario	GID efectivo = GID real

Salvo: bit SETUID o bit SETGID activos

IDENTIFICADORES DE SEGURIDAD DEL PROCESO II

Fichero ejecutable
SETUID activo

UID real = UID usuario	UID efectivo = UID propietario
GID real = GID usuario	GID efectivo = GID real

i-nodo

- Información de protección
- UID propietario
- GID propietario

Añadir permisos SETUID
$ chmod u+s /home/usuario/fich_ejecutable
Eliminar permisos SETUID
$ chmod u-s /home/usuario/fich_ejecutable

Fichero ejecutable
SETGID activo

UID real = UID usuario	UID efectivo = UID real
GID real = GID usuario	GID efectivo = GID propietario

i-nodo

- Información de protección
- UID propietario
- GID propietario

Añadir permisos SETGID
$ chmod g+s /home/usuario/fich_ejecutable
Eliminar permisos SETGID
$ chmod g-s /home/usuario/fich_ejecutable

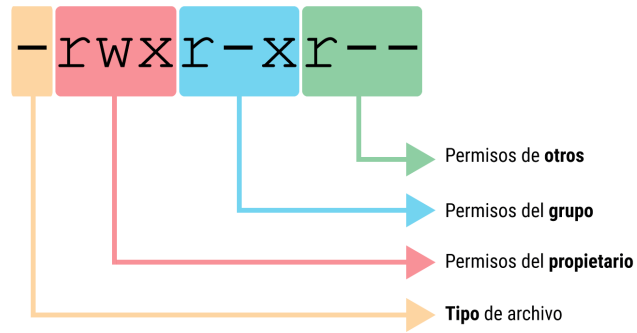
PROTECCIÓN UNIX SOBRE UN FICHERO O DIRECTORIO

datsi © UPM 2022

Por tanto, los procesos se ejecutan con

- UID real
- UID efectivo (se utiliza para los permisos)
- GID real
- GID efectivo (se utiliza para los permisos)

Reglas de protección



Bits SETUID y SETGID

- Si un proceso ejecuta un fichero con el SETUID activo, el UID efectivo del proceso pasa a ser el UID del propietario del fichero
- Si un proceso ejecuta un fichero con el SETGID activo, el GID efectivo del proceso pasa a ser el GID del propietario del fichero

Reglas de protección

- Si UID efectivo = 0 se concede el acceso (es superusuario)
- Si UID efectivo = UID del propietario, se utiliza el primer grupo de bits
- Si GID efectivo = GID del propietario, se utiliza el segundo grupo de bits
- En caso contrario, se utiliza el último grupo de bits

EJEMPLOS DE VOLCADO DEL MANDATO ls

datsi © UPM 2022

POLITÉCNICA

```
drwxr-x---   2 pepito prof    48 Dec 26  2001 News
drwxr-xr-x   2 pepito prof    80 Sep 29  2004 bin
lrwxrwxrwx   1 root root       3 Jan 23 18:34 lvremove -> lvm
lrwxrwxrwx   1 root root       3 Jan 23 18:34 lvrename -> lvm
drwxrwxrwt  16 root root    1928 Apr  9 20:26 tmp
-rwxr-xr-x   1 root root    2436 Dec 26  2001 termwrap
-rwsr-xr-x   1 root root   22628 Jan  5 10:15 mount.cifs
```

Carácter inicial:

-	fichero normal
d	directorio
l	enlace simbólico
b	dispositivo de bloques
c	dispositivo de caracteres
p	FIFO
s	socket UNIX

Bit x:

- **Si aparece una “s” significa que está activo el SETUID o SETGID, según la posición.**
- **Si aparece una “t” en un directorio se permite crear y borrar entradas con el UID efectivo del proceso (se usa para directorios temporales). No se podrán borrar las entradas de otros usuarios.**

ESTABLECER PERMISOS ESPECIALES

datsi © UPM 2022

POLITÉCNICA

- - - - - - - - -	0	Predeterminado, sin permisos especiales. No se requiere indicar.
- - - - - - - - t	1	Bit de persistencia, sticky bit
- - - - - s - - -	2	Bit sgid de grupo
- - - - - s - - t	3	Bit sgid y sticky
- - s - - - - - -	4	Bit suid
- - s - - - - - t	5	Bit suid y sticky
- - s - - s - - -	6	Bit suid y sgid
- - s - - s - - t	7	Bit suid, sgid y sticky

```
#> ls -l
/usr/prog -r-x--x--x 24 root root 4096 sep 25 18:14 prog
#> chmod 04511 /usr/prog
#> ls -l
/usr/prog -r-s--x--x 24 root root 4096 sep 25 18:14 prog
```



POLITÉCNICA

SERVICIOS DEL SERVIDOR DE FICHEROS

VIDA DEL FICHERO

© UPM 2022

- **Se crea.**
  - **Se abre → se genera un fd (descriptor de fichero).**
    - Se escribe y lee (el fichero puede crecer).
  - **Se cierra.**
- **Se borra.**

Para trabajar con un fichero hay que definir una sesión con las llamadas open y close

Las operaciones de lectura y escritura se hacen a partir del puntero de posición, que queda modificado

SEMÁNTICA DE COUTILIZACIÓN

Semántica de coutilización: especifica el efecto de varios procesos accediendo de forma simultánea al mismo fichero

Existen diferentes tipos de semánticas

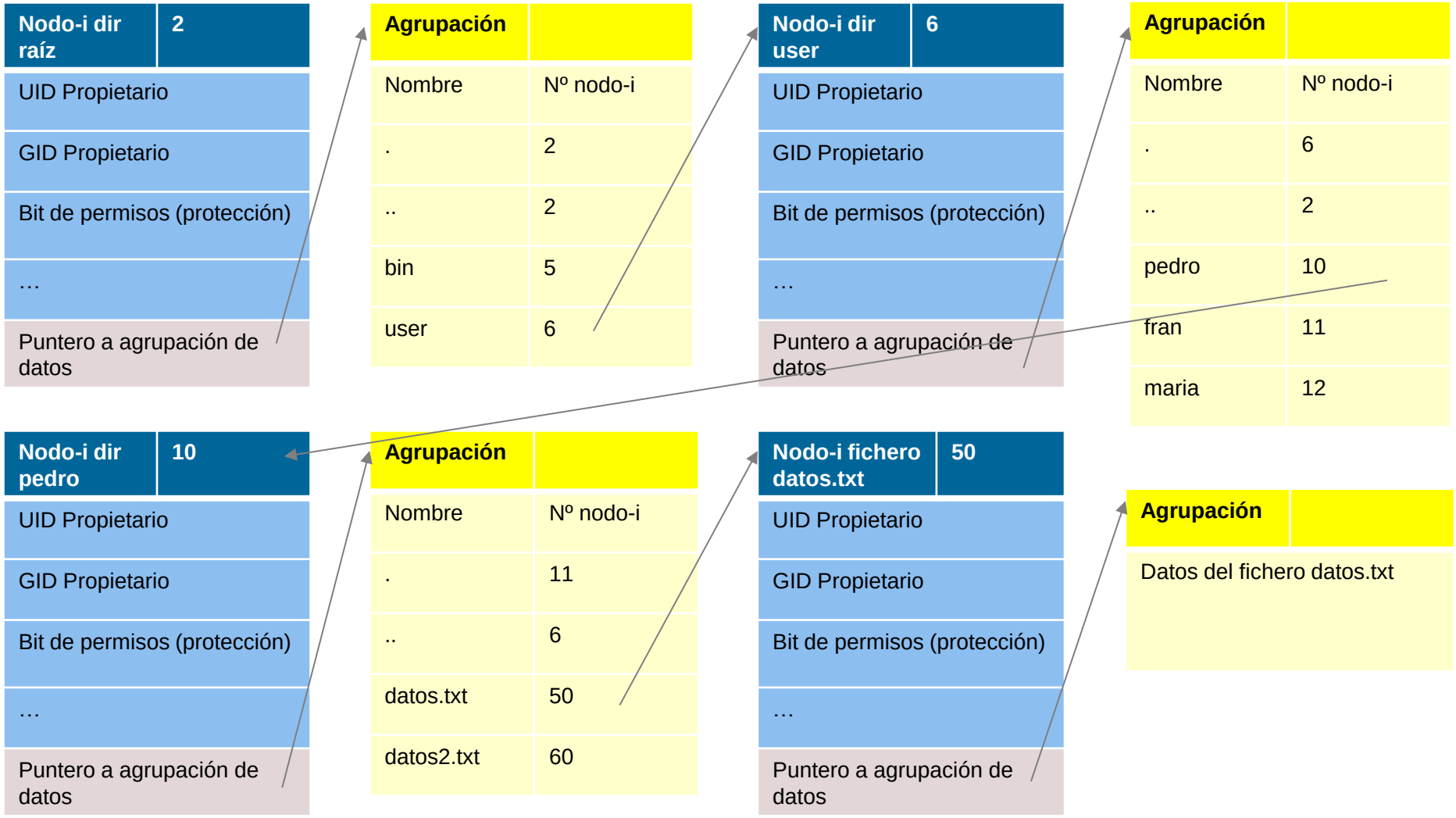
Semántica de coutilización UNIX

- **Los procesos pueden compartir ficheros de forma simultánea**
- **El puntero se crea en el open**
- **El puntero se comparte cuando se hereda o duplica el descriptor**
- **Las escrituras son inmediatamente visibles para todos los procesos con el fichero abierto**
- **Los datos se escriben en orden de llegada al SO (si se quiere un orden específico => usar cerrojos –mecanismo de sincronización-)**
- **La coutilización afecta a los metadatos (atributos del i-nodo)**

datsi © UPM 2022

Servicio open del fichero /user/pedro/datos.txt

Se recorren las tablas de directorio hasta encontrar el nombre buscado
Se analizan los permisos que están almacenados en los i-nodos
Se parte del directorio raíz o del directorio de trabajo

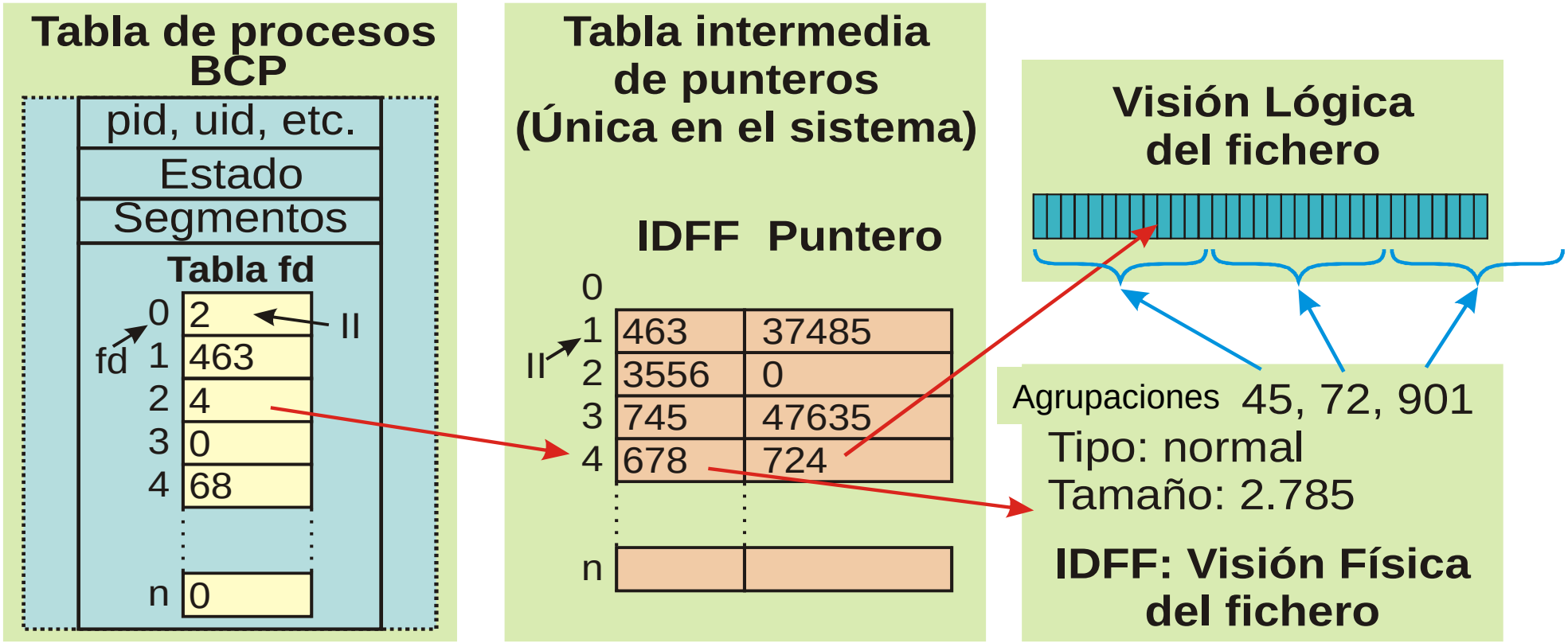


SERVICIOS OPEN II

datsi © UPM 2022

- La tabla fd existe por proceso y contiene Identificadores Intermedios (II).
- En memoria hay una tabla intermedia de punteros única que relaciona los II con los punteros de posición para escritura y lectura y con los IDFF.
- El valor II = 0 indica que ese elemento de la tabla de fd está libre.

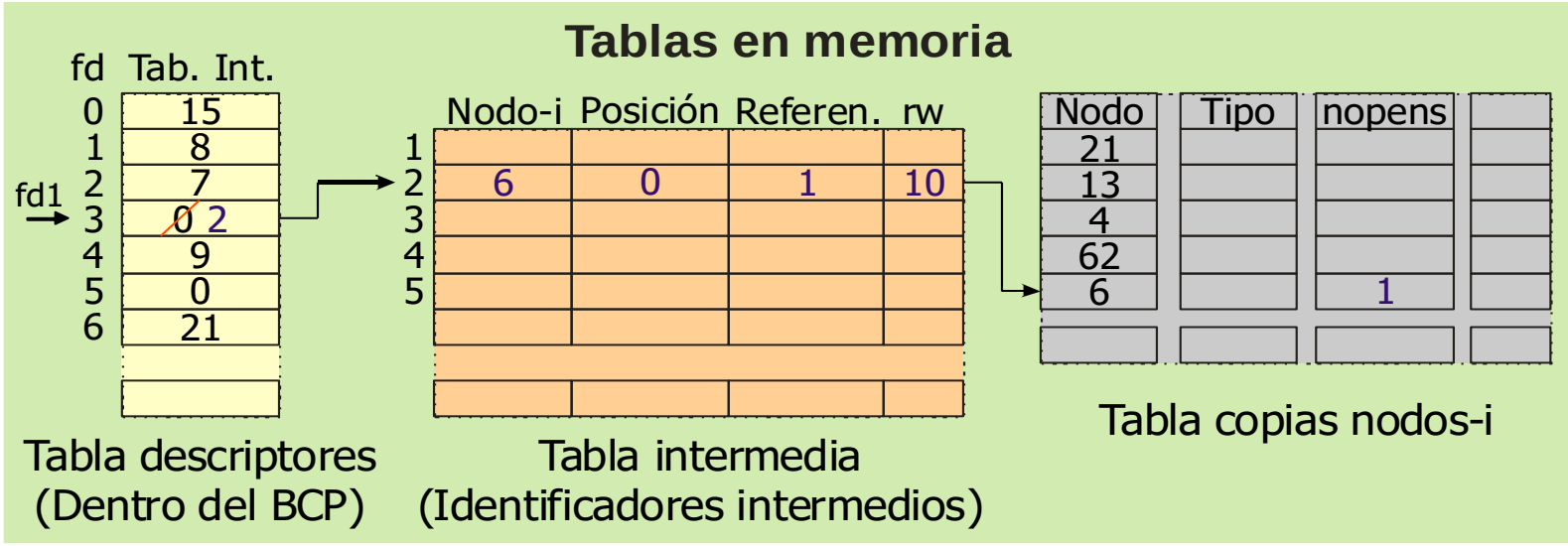
(p.e. el programa usa fd = 2 → II = 4 → IDF = 678; Posición = 724)



SERVICIO OPEN III

datsi © UPM 2022

- Se añade una entrada en la tabla de ficheros abiertos que existe en el BCP del proceso
- Se añade una entrada en la tabla intermedia de punteros, poniendo el puntero a 0 y las referencias a 1. (Al crear un hijo se incrementará el campo de referencias)
- Si el fichero no estaba abierto se copia el nodo_i en la tabla de nodos_i residente en memoria y se añade el campo nopens (nº de opens). Se hace nopens = 1
- Si el fichero ya estaba abierto se hace nopens = nopens + 1, para llevar la cuenta de cuántas veces está abierto el fichero



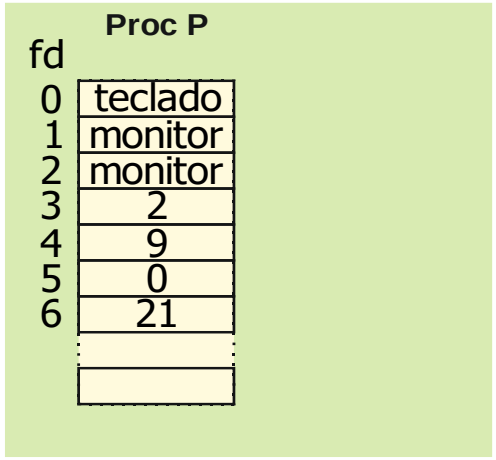
OPEN IV – APERTURA DE UN FICHERO

datsi © UPM 2022

POLITÉCNICA

Servicio: (man 2 open)

```
int open(char *name, int flags{, mode_t mode});
```



Argumentos:

- *name:* **Nombre del fichero**
- *flags*: **Opciones de apertura:**
  - **O_RDONLY: Sólo lectura**
  - **O_WRONLY: Sólo escritura**
  - **O_RDWR: Lectura y escritura**
  - **O_APPEND: Se accede siempre al final del fichero**
  - **O_CREAT: Si existe no tiene efecto. Si no existe lo crea**
  - **O_TRUNC: Trunca a cero si se abre para escritura**
- *mode*: **Bits de permiso para el fichero. Valen sólo cuando se crea (O_CREAT)**

Devuelve: Un descriptor de fichero o -1 si hay error o señal (`errno = EINTR`*)

Descripción:

- **Se comprueban los derechos de acceso de todo el camino especificado (absoluto o relativo)**
- **Se asigna descriptor de fichero en la tabla fd del BCP, una entrada en la tabla intermedia y se copia el nodo_i a memoria o incrementa su nº opens**

Ejemplos:

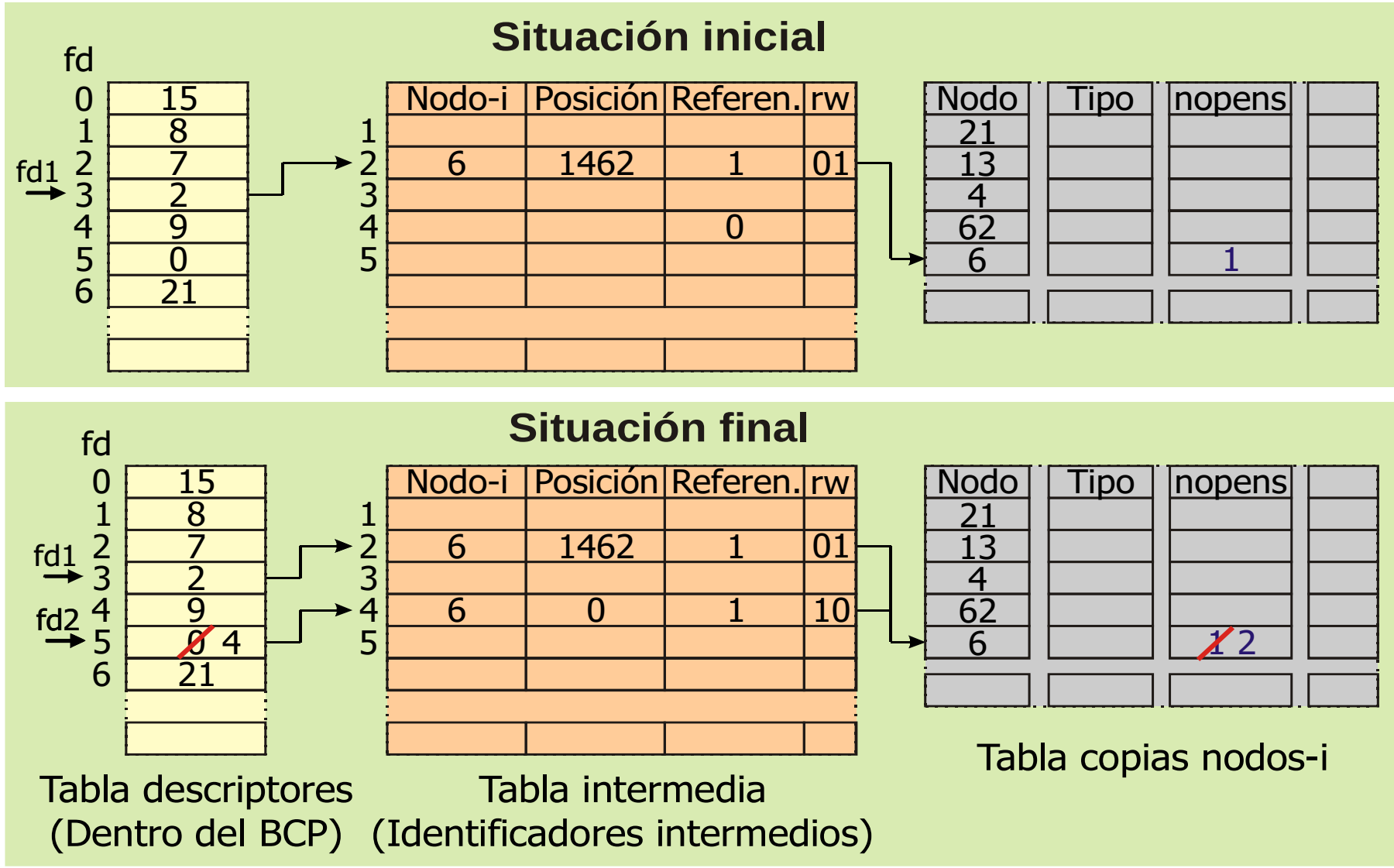
```
fd = open("/home/juan/dat.txt", O_RDONLY);
fd = open("/home/juan/dat.txt", O_WRONLY|O_CREAT|O_TRUNC, 0640);
```

datsi © UPM 2022

POLITÉCNICA

El mismo proceso vuelve a abrir el mismo fichero datos.txt de nodo-i = 6

```
fd2 = open("datos.txt",O_RDONLY);
```

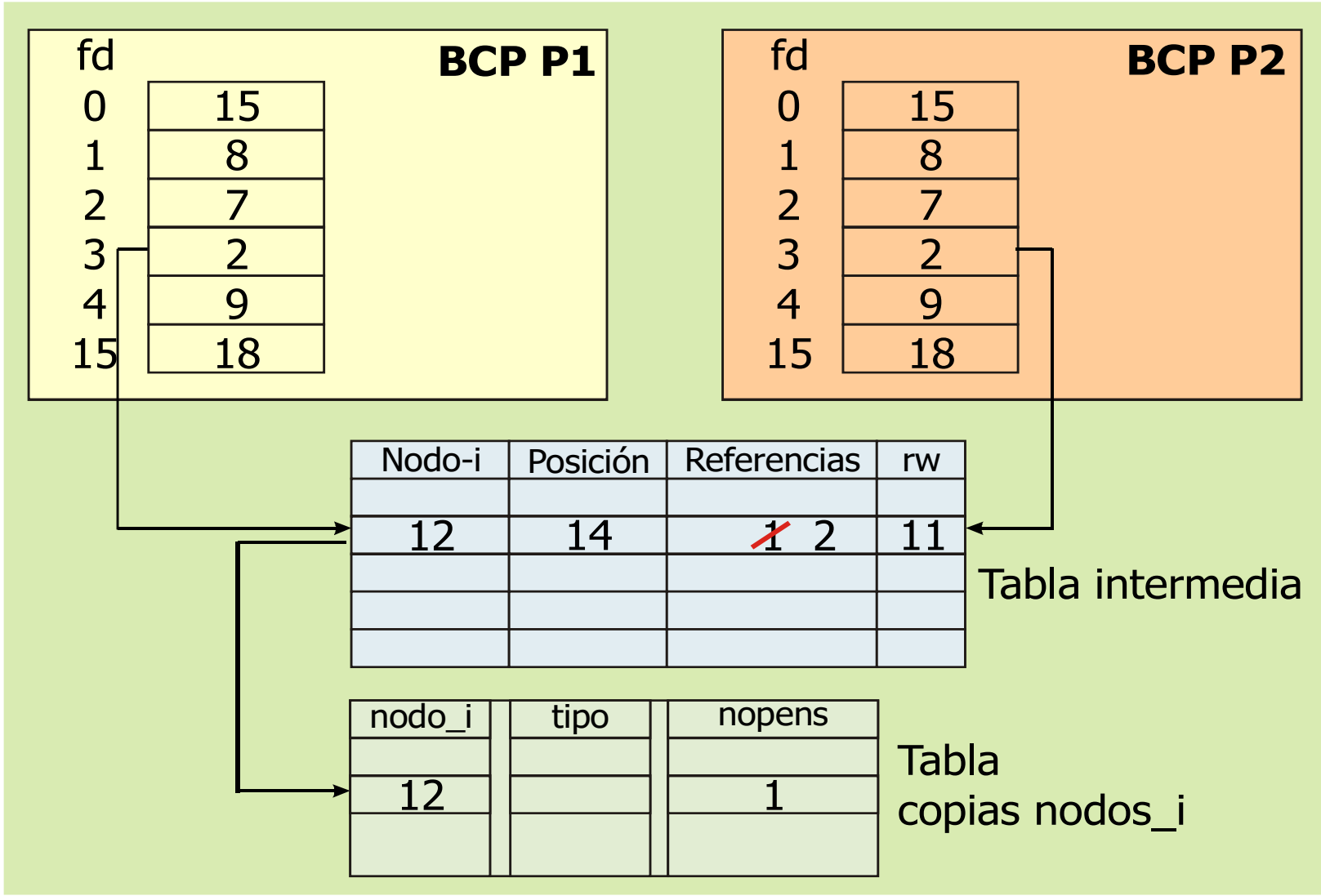


GESTIÓN DE PUNTEROS (FORK)

datsi © UPM 2022

POLITÉCNICA

```
pid = fork(); /*crea proceso hijo */
```



CREAT. CREA Y ABRE PARA ESCRITURA UN FICHERO

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int creat(char *name, mode_t mode);
```

Argumentos:

- *name:* Nombre de fichero
- *mode*: Bits de permiso para el fichero (en caso de nueva creación)

Devuelve:

- Devuelve un descriptor de fichero o -1 si fracasa

Descripción:

- Se comprueban los derechos de acceso de todo el camino
- El fichero se abre sólo para escritura
- Si no existe crea un fichero vacío
  - UID_propietario = UID_efectivo
  - GID_propietario = GID_efectivo
  - Con los permisos indicados enmascarados (`mode & ~umask`)
- Si existe lo trunca (tamaño = 0) sin cambiar los bits de permiso
- Se asigna descriptor de fichero en la tabla fd del BCP, una entrada en la tabla intermedia y se copia el nodo_i en memoria o incrementa su nº opens

Ejemplos que producen el mismo resultado:

```
fd = creat("datos.txt", 0751);   ¿Tienen sentido estos permisos?
fd = open("datos.txt", O_WRONLY | O_CREAT | O_TRUNC, 0751);
```

READ. LECTURA DE UN FICHERO*

datsi © UPM 2022

Servicio:

```
ssize_t read(int fd, void *buf, size_t n_bytes);
```

Argumentos:

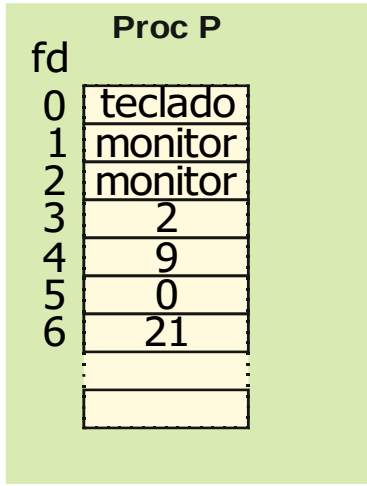
- ***fd:*** **descriptor de fichero**
- ***buf*****: zona donde almacenar los datos**
- ***n_bytes*****: número de bytes a leer**

Devuelve:

- **Número de bytes realmente leídos o -1 si fracasa**

Descripción:

- **Transfiere n_bytes como máximo**
- **Puede leer menos datos de los solicitados si se llega al fin de fichero.**
- **El servicio puede fracasar por una señal, retornando -1. NO ES ERROR**
- **Después de la lectura se incrementa el puntero del fichero con el número de bytes realmente transferidos**
- **Si retorna 0, indica final de fichero**
- **En ficheros especiales (como pipes) se queda bloqueado hasta que hay datos**



*: Objeto asociado a un descriptor de fichero, en general

WRITE. ESCRITURA DE UN FICHERO *

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
ssize_t write(int fd, void *buf, size_t n_bytes);
```

Argumentos:

- ***fd*: descriptor de fichero**
- ***buf*: zona de datos a escribir**
- ***n_bytes*: número de bytes a escribir**

Devuelve:

- **Número de bytes realmente escritos -1 si fracasa**

Descripción:

- **Transfiere *n_bytes* o menos**
- **Si se rebasa el fin de fichero el fichero aumenta de tamaño**
- **Puede escribir menos datos de los solicitados si se llega al tamaño máximo del fichero o se rebasa algún limite de implementación del sistema operativo**
- **El servicio puede fracasar por una señal, retornando -1 . NO ES ERROR**
- **Después de la escritura se incrementa el puntero del fichero con el número de bytes realmente transferidos**



*: Objeto asociado a un descriptor de fichero, en general

datsi © UPM 2022

POLITÉCNICA

```
int total;
n = read(d, &total, sizeof(int)); /*Se lee un entero */
total = 1246; /*Si sizeof(int) = 4 total ? 00 00 04 DE */
n = write(d, &total, sizeof(int));
   /*Se escriben los cuatro bytes 00, 00, 04 y DE, no los cuatro caracteres ASCII 1,
   2, 4 y 6 */
float m[160];
n = read(d, m, sizeof(float)*160);
/*Se leen hasta 160 float, lo que pueden ser 4*160 bytes */
typedef struct registro{
   int identificador;
   float edad, altura, peso;
} registro;
registro individuo[500];
n = read(d, individuo, 500*sizeof(registro));
estadistica_ciega(individuo, n/sizeof(registro));
```

CLOSE. CIERRE DE UN DESCRIPTOR DE FICHERO

datsi © UPM 2022

Servicio:

```
int close(int fd);
```

Argumentos:

- ***fd:*** **descriptor de fichero**

Devuelve:

- **0 o -1 si error**

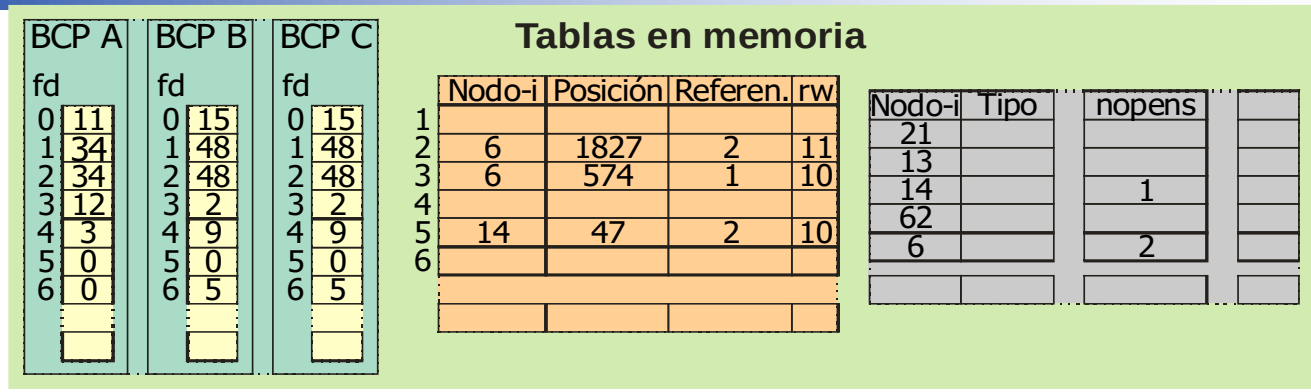
Descripción:

- **El proceso pierde la asociación entre el descriptor y el fichero (u objeto correspondiente)**
- **Se decrementa el número de referencias en la tabla intermedia. Si referencias = 0 entonces:**
  - **Se libera la correspondiente entrada de la tabla intermedia y**
  - **Se decrementa nopens (número de duplicados) de la tabla copias nodos_i**
  - **Si nopens = 0 se libera la entrada en la tabla copias nodos_i**

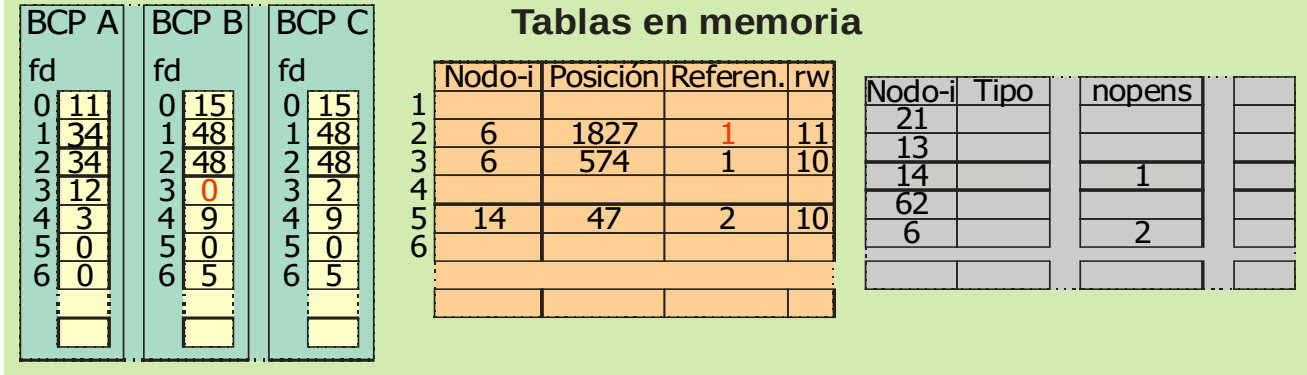
CLOSE. CIERRE DE UN DESCRIPTOR DE FICHERO

datsi © UPM 2022

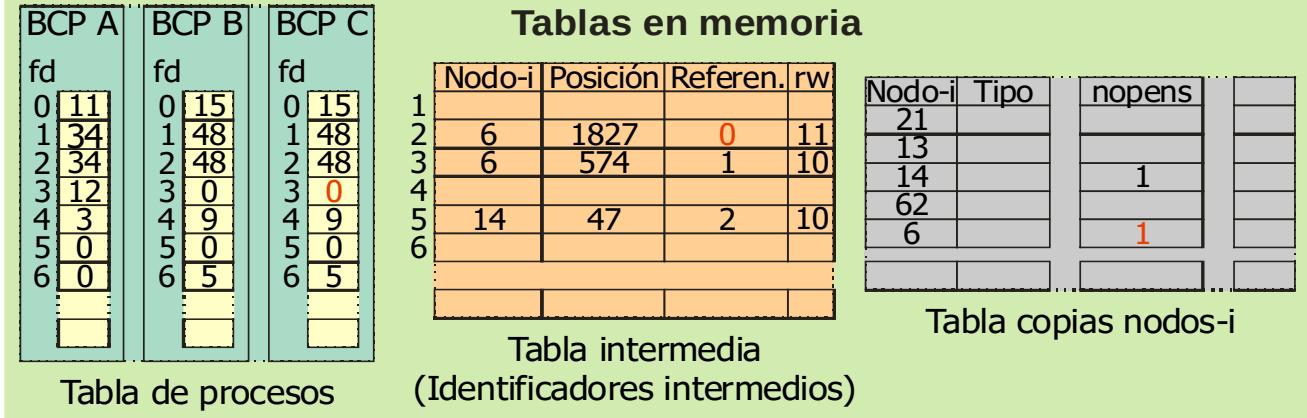
POLITÉCNICA



Proceso B ejecuta: `close (3);`



Proceso C ejecuta: `close (3);`



EJEMPLO FICHEROS. COPIA UN FICHERO EN OTRO I

Los nombres de los ficheros se pasan como argumentos

```
#define BUFSIZE 4096
int main(int argc, char **argv) {
   int fd_ent, fd_sal, n_read;
   char buffer[BUFSIZE];
   //Abre fichero origen o entrada
   fd_ent = open(argv[1], O_RDONLY);
   if (fd_ent < 0) {
       perror("open"); //transforma errno en un mensaje de error
       exit(1);
   }
   //Crea fichero de destino o salida
   fd_sal = creat(argv[2], 0666);
   if (fd_sal < 0) {
       close(fd_ent);
       perror("creat");
       exit(1);
   }
```

EJEMPLO FICHEROS. COPIA UN FICHERO EN OTRO II


```
    /* bucle de lectura del fichero de entrada */
    while ((n_read = read(fd_ent, buffer, BUFSIZE)) > 0) {
        /* escribir el buffer al fichero de salida */
        if (write(fd_sal, buffer, n_read) < n_read) {
            perror("write");
            close(fd_ent);
            close(fd_sal);
            exit(1);
        }
    }
    close(fd_ent);
    close(fd_sal);
    if (n_read < 0) {
        perror("read");
        exit(1);
    }
    return 0;
}
```

LSEEK. MODIFICA EL PUNTERO DE POSICIÓN

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
off_t lseek(int fd, off_t offset, int whence);
```

Argumentos:

- ***fd*: Descriptor de fichero**
- ***offset*: desplazamiento (positivo o negativo)**
- ***whence*: base del desplazamiento**

Devuelve:

- **La nueva posición del puntero o -1 si error (terminal, pipe, socket o FIFO `errno=EPIPE`).**

Descripción:

- **Coloca el puntero de acceso asociado a *fd***
- **La nueva posición, que no puede ser negativa, se calcula según el valor de *whence*:**
  - **SEEK_SET: posición = offset**
  - **SEEK_CUR: posición = posición actual + offset**
  - **SEEK_END: posición = final del fichero + offset**
- **Nos podemos salir del tamaño del fichero (no se aumenta el tamaño, salvo que se haga una escritura posterior)**

LSEEK. EJEMPLOS

datsi © UPM 2022

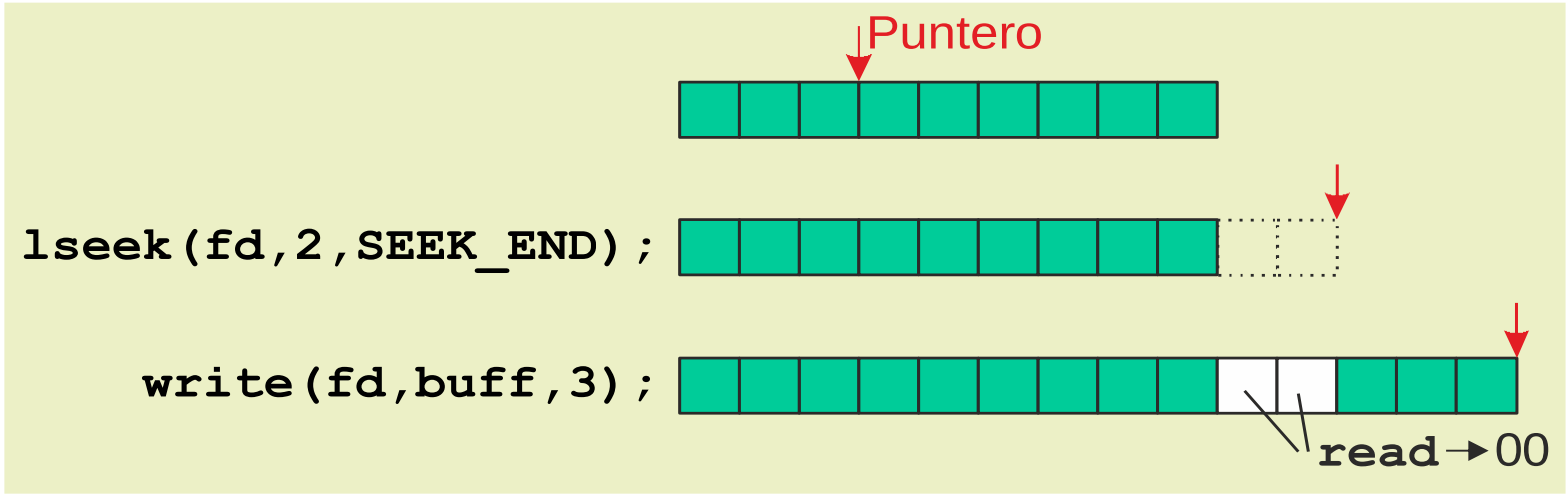
- Tamaño de fichero:

```
tam = lseek(fd,0,SEEK_END);
```
← ¿qué pasa si hay huecos?

- Posición actual:

```
pos = lseek(fd,0,SEEK_CUR);
```

- Creación de huecos:



- Al escribir es cuando se aumenta el tamaño real
- El tamaño ocupado pude ser < que el real (agrupaciones de hueco no asignados)
- Si se lee del hueco no escrito se obtienen ceros

DUP. DUPLICA UN DESCRIPTOR DE FICHERO

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int dup(int fd);
```

Argumentos:

- ***fd*: descriptor de fichero**

Devuelve:

- **Un descriptor de fichero que comparte todas las propiedades del fd o -1 si error**

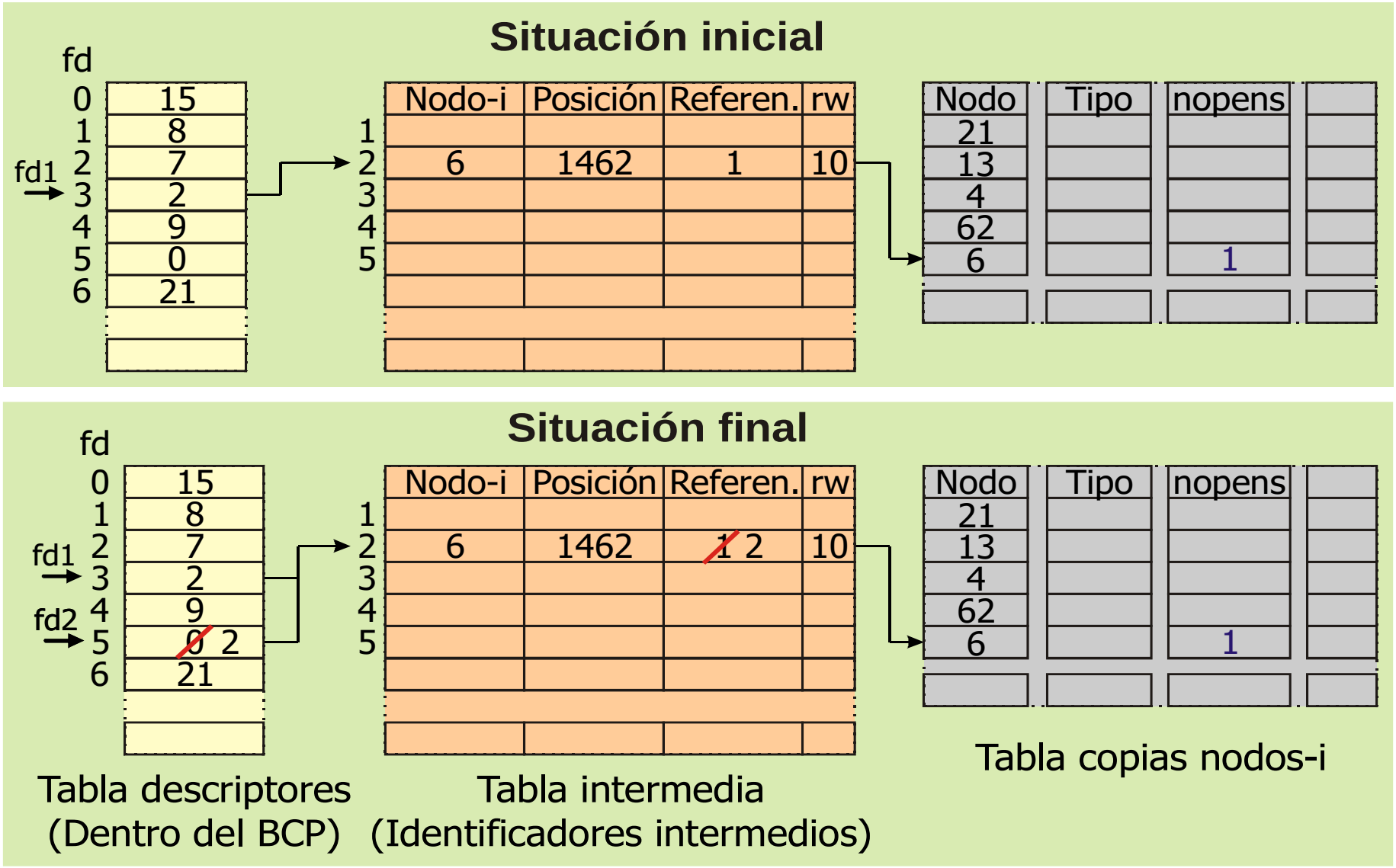
Descripción:

- **Crea un nuevo descriptor de fichero que tiene en común con el anterior:**
  - **Accede al mismo fichero**
  - **Comparte el mismo puntero de posición**
  - **El modo de acceso es idéntico**
- **El nuevo descriptor tendrá el menor valor numérico posible**
- **Se incrementa en 1 el número de referencias (tabla intermedia)**

datsi © UPM 2022

POLITÉCNICA

```
fd2= dup(fd1);
```



DUP2. DUPLICA UN DESCRIPTOR DE FICHERO

datsi © UPM 2022

Servicio:

```
int dup2(int oldfd, int newfd);
```

Argumentos:

- ***oldfd*: descriptor de fichero existente**
- ***newfd:* nuevo descriptor de fichero**

Devuelve:

- **El nuevo descriptor de fichero o -1 si error**

Descripción:

- **Crea un nuevo descriptor de fichero, cuyo número es *newfd,* que tiene en común con *oldfd*:**
  - **Accede al mismo fichero**
  - **Comparte el mismo puntero de posición**
  - **El modo de acceso es idéntico**
- **Si *newfd* estaba abierto, lo cierra antes de realizar el duplicado**
- **Se incrementa en 1 el número de referencias (tabla intermedia)**

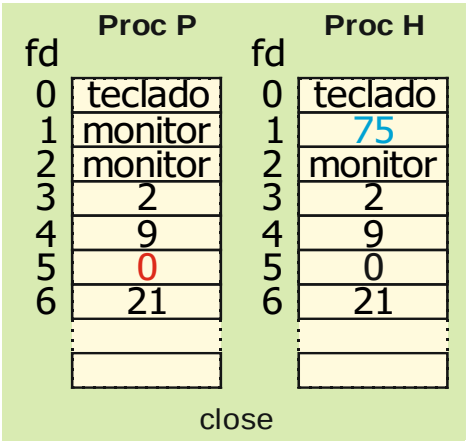
EJEMPLO: REDIRECCIONES

datsi © UPM 2022

POLITÉCNICA

Programa que ejecuta: ls > fichero

```
int main(void){
   pid_t pid;
   int status, fd;
   fd = open("fichero", O_WRONLY|O_CREAT|O_TRUNC, 0666);
   if (fd < 0) {perror("open"); exit(1);}
   pid = fork();
   switch(pid) {
       case -1:        /* error */
           perror("fork"); exit(1);
       case 0:         /* proceso hijo ejecuta "ls" */
           close(1);
           dup(fd);    /* el descriptor se copia en el 1 */
           close(fd);
           execlp("ls","ls",NULL);
           perror("execlp"); exit(1);
       default:        /* proceso padre */
           close(fd);
           while (pid != wait(&status));
   }
    return 0;
}
```



EJEMPLO: REDIRECCIONES

Programa que ejecuta: ls > fichero

```
int main(void){
   pid_t pid;
   int status, fd;
   fd = open("fichero", O_WRONLY|O_CREAT|O_TRUNC, 0666);
   if (fd < 0) {
       perror("open"); exit(1);
   }
   pid = fork();
   switch(pid) {
       case -1:            /* error */
           perror("fork"); exit(1);
       case 0:             /* proceso hijo ejecuta "ls" */
           dup2(fd, 1); /* el descriptor se copia en el 1 */
           close(fd);
           execlp("ls","ls",NULL);
           perror("execlp");
           exit(1);
       default:            /* proceso padre */
           close(fd);
           while (pid != wait(&status));
   }
    return 0;
}
```



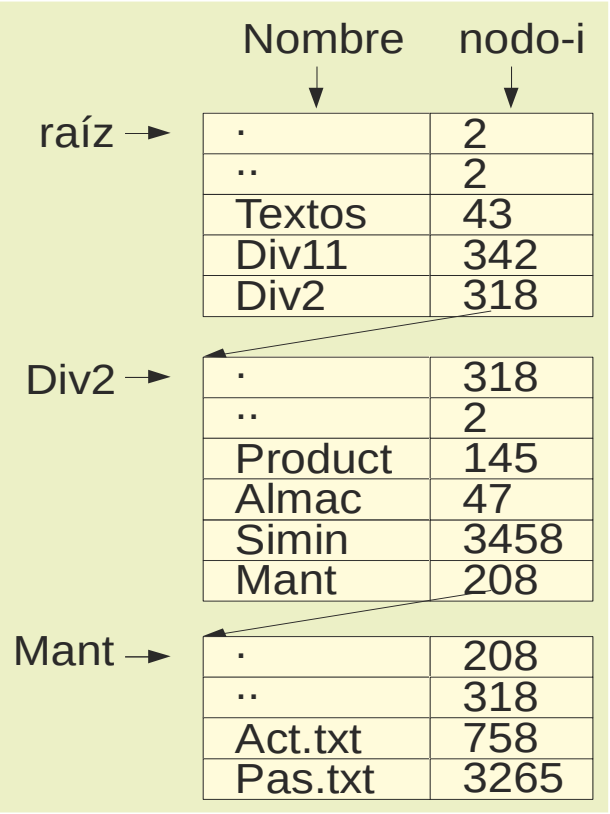
SERVICIOS UNIX SOBRE DIRECTORIOS

OPENDIR. ABRE UN DIRECTORIO

datsi © UPM 2022

El fichero directorio es un fichero de registros de tipo `struct dirent`. Esta estructura se define en `dirent.h` y es dependiente de la implementación, pero incluye:

- **`ino_t d_ino`: número de nodo_i**
- **`char d_name[]`: nombre del fichero (de tamaño varible)**



Servicio:

```
DIR *opendir(char *dirname);
```

Argumentos:

- ***dirname:*** **Nombre del directorio**

Devuelve:

- **Un puntero para utilizarse en `readdir()`, `rewindir()` o `closedir()` y `NULL` si fracasa**

Descripción:

- **Abre un directorio y se coloca en el primer registro o entrada del mismo**

Permisos:

- **Permiso de lectura de `dirname`**

READDIR. LECTURA DE UN REGISTRO DE DIRECTORIO

datsi © UPM 2022

Servicio:

```
struct dirent *readdir(DIR *dirp);
```

Argumentos:

- ***dirp:*** **puntero devuelto por `opendir()`**

Devuelve:

- **Un puntero a un objeto del tipo `struct dirent` que representa un registro de directorio o `NULL` si hubo error o se ha llegado al final del directorio**

Descripción:

- **Devuelve el siguiente registro del directorio `dirp`**
- **Avanza el puntero al siguiente registro**

CLOSEDIR. CIERRA UN DIRECTORIO

datsi © UPM 2022

Servicio:

```
int closedir(DIR *dirp);
```

Argumentos:

- ***dirp:*** **puntero devuelto por `opendir()`**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Cierra la asociación entre `dirp` y la secuencia de entradas de directorio**

REWINDIR. INICIALIZA EL PUNTERO DEL DIRECTORIO

datsi © UPM 2022

Servicio:

```
void rewindir(DIR *dirp);
```

Argumentos:

- `dirp` puntero devuelto por `opendir()`

Descripción:

- Sitúa el puntero de posición del directorio en el primer registro o entrada

PROGRAMA QUE LISTA UN DIRECTORIO

datsi © UPM 2022

POLITÉCNICA

```
#define MAX_BUF   256
int main(int argc, char **argv){
    DIR *dirp;
    struct dirent *dp;
    char buf[MAX_BUF];
    /* imprime el directorio actual */
    getcwd(buf, MAX_BUF);
    printf("Directorio actual: %s\n", buf);
    /* abre el directorio pasado como argumento */
    dirp = opendir(argv[1]);
    if (dirp == NULL)  {
        fprintf(stderr,"No puedo abrir %s\n", argv[1]);
    } else {
        /* lee entrada a entrada */
        while ((dp = readdir(dirp)) != NULL){
                printf("%ld\n", dp->d_ino);
                printf("%s\n", dp->d_name);
         }
        closedir(dirp);
    }
   return 0;
}
```

MKDIR. CREA UN DIRECTORIO

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int mkdir(char *name, mode_t mode);
```

Argumentos:

- ***name:*** **nombre del directorio**
- ***mode:*** **bits de protección**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Crea un directorio de nombre `name` y permisos `mode & ~umask`**
- **UID_dueño directorio = UID_efectivo proceso**
- **GID_dueño directorio = GID_efectivo proceso**

Permisos

- **Escritura en directorio donde se añade `name`**

RMDIR. BORRA UN DIRECTORIO

datsi © UPM 2022

Servicio:

```
int rmdir(char *name);
```

Argumentos:

- *name:* **nombre del directorio**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Borra el directorio si está vacío**
- **Si el directorio no está vacío no se borra**

Permisos

- **Escritura en directorio de donde se elimina `name`**

CHDIR. CAMBIA EL DIRECTORIO ACTUAL

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int chdir(char *name);
```

Argumentos:

- ***name:*** **nombre de un directorio**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Modifica el directorio actual, aquel a partir del cual se forman los nombre relativos**

Permisos:

- **Requiere permisos de búsqueda (x) para `name`**

RENAME. CAMBIA EL NOMBRE DE UN FICHERO

datsi © UPM 2022

Servicio:

```
int rename(char *old, char *new);
```

Argumentos:

- ***old:*** **nombre de un fichero existente**
- ***new:*** **nuevo nombre del fichero**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Cambia el nombre del fichero *old*. El nuevo nombre es *new***

Permisos:

- **Permiso de escritura en los directorios que contienen a `old` y a `new`.**

GETCWD. OBTIENE EL NOMBRE DEL DIRECTORIO ACTUAL

datsi © UPM 2022

Servicio:

```
char *getcwd(char *buf, size_t size);
```

Argumentos:

- ***buf:*** **dirección del espacio donde se va a almacenar el nombre del directorio actual**
- ***size:*** **longitud en bytes de dicho espacio**

Devuelve:

- **Puntero a *buf* o *NULL* si error (p.e. si el nombre ocupa más de `size`)**

Descripción:

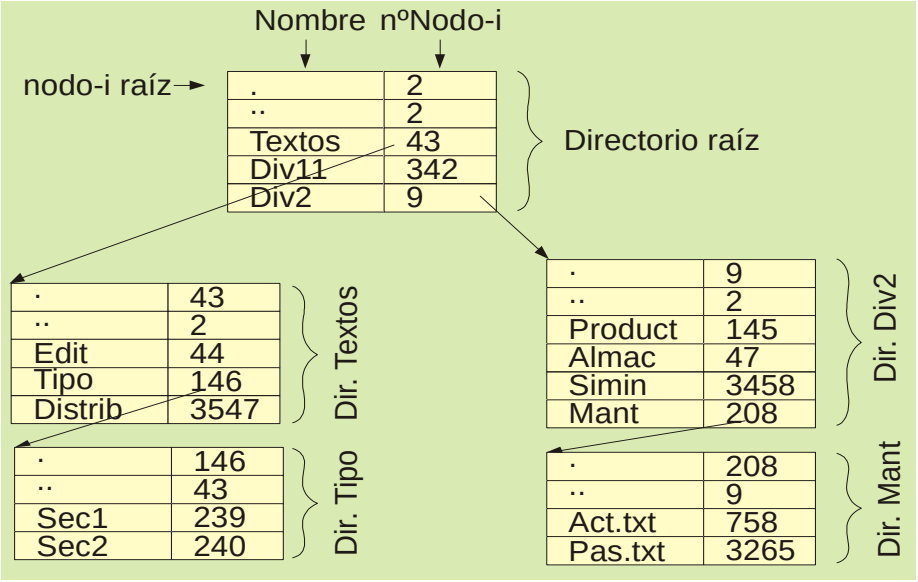
- **Obtiene el nombre del directorio actual**

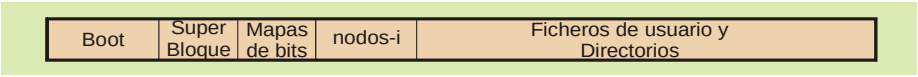
ENLACES

datsi © UPM 2022

Permite que dos o más nombres hagan referencia al mismo fichero

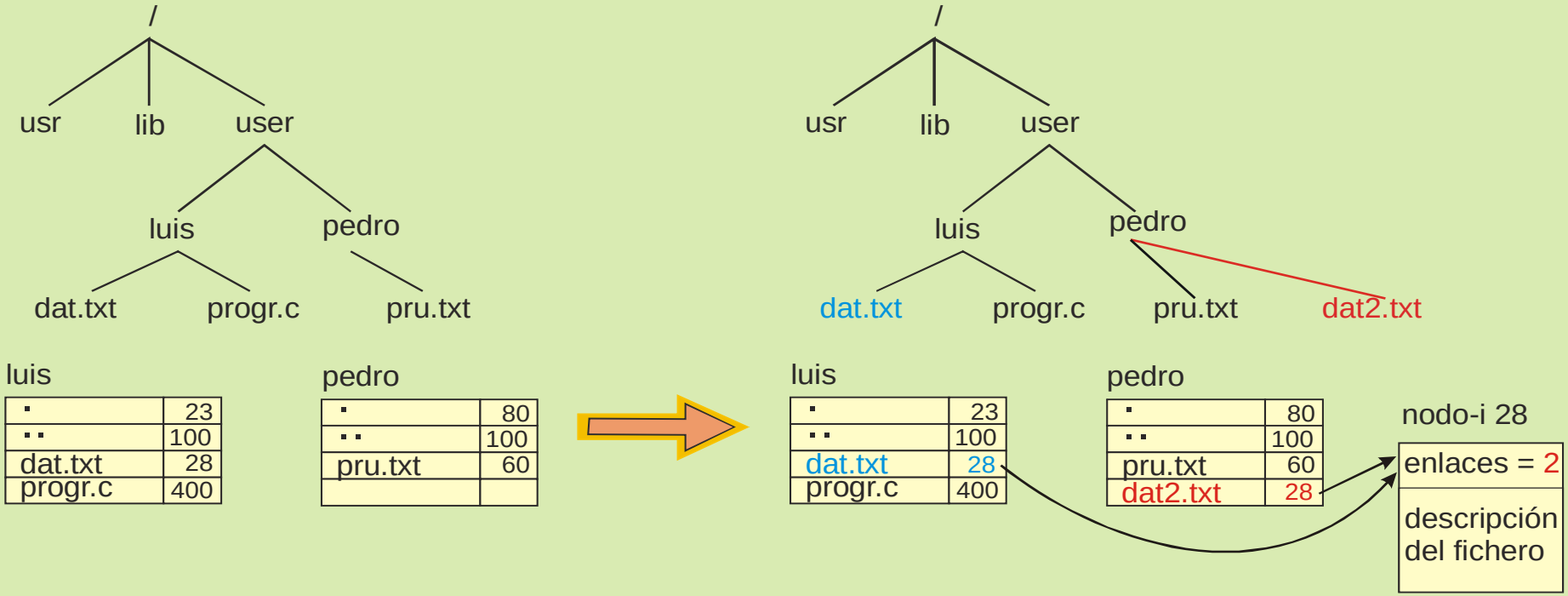
- **Enlace físico**
  - **El fichero sólo se elimina cuando se borran todos sus nombres**
  - **Contador en el nodo-i del fichero. Necesario para saber cuándo se puede borrar un fichero. Con contador = 0 se borra el fichero**
  - **Sólo se permiten enlazar ficheros del mismo volumen**
  - **No perder de vista que los nombres . y .. también cuentan. ¿Cuántos enlaces tiene un directorio?**
- **Enlace simbólico**
  - **El fichero se elimina cuando el contador de enlaces físicos = 0. Si permanece el enlace simbólico se provocará un error al tratar de abrirlo**
  - **Se puede hacer con ficheros y directorios**
  - **Se puede hacer entre ficheros de diferentes volúmenes**





ENLACE FÍSICO

- **Dos entradas de directorio al mismo nodo-i**
- **Solamente para ficheros de un mismo SF**
- **Hay que incrementar el contador de enlaces en el nodo-i**



Mandato: `ln /user/luis/dat.txt /user/pedro/dat2.txt`

Servicio: `link ("/user/luis/dat.txt","/user/pedro/dat2.txt");`

LINK. CREA UNA ENTRADA (ENLACE) DE DIRECTORIO

datsi © UPM 2022

Servicio:

```
int link(char *existing, char *new);
```

Argumentos:

- ***existing:*** **nombre del fichero existente**
- ***new:*** **nombre de la nueva entrada que será un enlace al fichero existente. No puede existir previamente**

Devuelve: Cero o -1 si fracasa

Descripción:

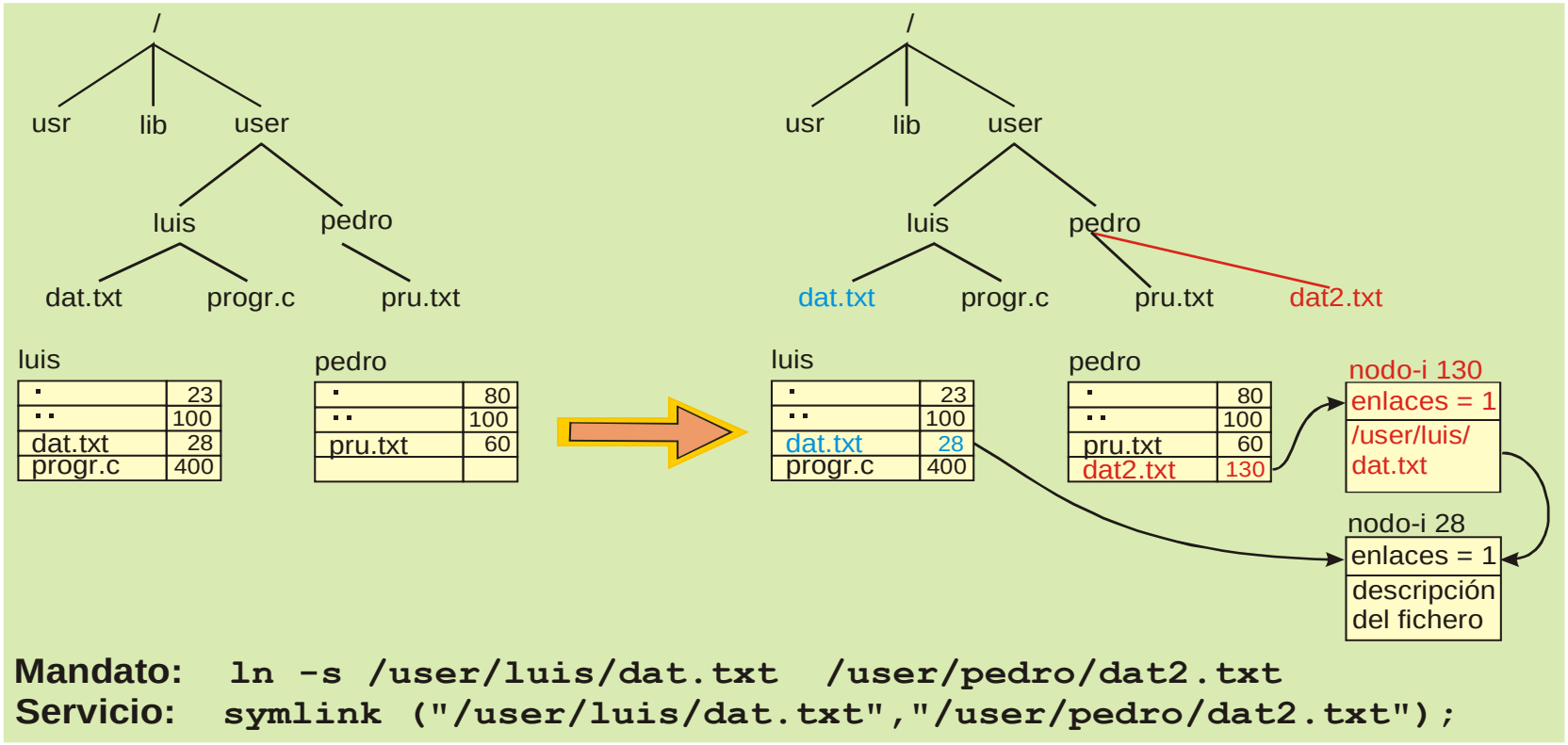
- **Crea un nuevo enlace físico para un fichero existente**
- **Requiere permisos de escritura en el directorio donde esté `new`**
- **El sistema no registra cuál es el enlace original**
- **`existing` no debe ser el nombre de un directorio salvo que se tenga privilegio suficiente y la implementación soporte el enlace de directorios. El enlace de directorios puede dar lugar a ciclos en el árbol de nombres**
- **Las entradas . y .. son enlaces físicos de directorio, pero controlados por el sistema operativo**

El `open` y `creat` también pueden crear una entrada de directorio

ENLACE SIMBÓLICO

datsi © UPM 2022

- Para ficheros y directorios de un mismo o diferentes SF
- Se utiliza un nodo-i para almacenar la referencia
- No se incrementa el contador de enlaces del fichero
- Para abrir un fichero se comprueban los derechos del camino definido en el enlace simbólico



Mandato: `ln -s /user/luis/dat.txt /user/pedro/dat2.txt`

Servicio: `symlink ("/user/luis/dat.txt","/user/pedro/dat2.txt");`

SYMLINK. CREA UNA ENTRADA DE DIRECTORIO

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int symlink(char *existing, char *new);
```

Argumentos:

- ***existing:*** **nombre del fichero existente (no se comprueba que exista)**
- ***new:*** **nombre de la nueva entrada que será un enlace al fichero existente. Es necesario tener permisos de escritura en el directorio**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Crea un nuevo enlace simbólico para un fichero o directorio existente**
- **Requiere permiso de escritura en el directorio donde esté `new`**
- **El fichero o directorio puede desaparecer dejando el enlace "colgado"**
- **Para abrir un fichero se comprueban también los derechos de todo el camino definido en el enlace simbólico**
- **Una vez creado el enlace, la gran mayoría de los servicios que utilicen `new` como argumento se realizarán realmente sobre `existing`. Por ejemplo el `chmod` se hará sobre `existing`**

UNLINK. ELIMINA UNA ENTRADA DE DIRECTORIO

datsi © UPM 2022

Servicio:

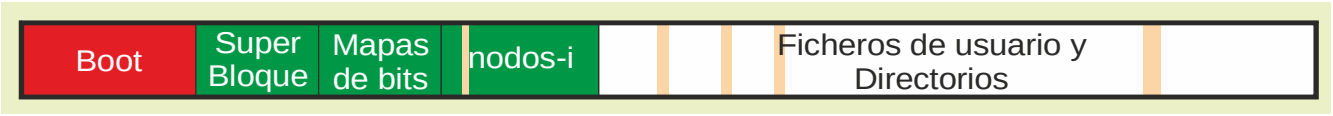
```
int unlink(char *name);
```

Argumentos:

- ***name:*** **nombre de fichero**

Devuelve:

- **Cero o -1 si error**



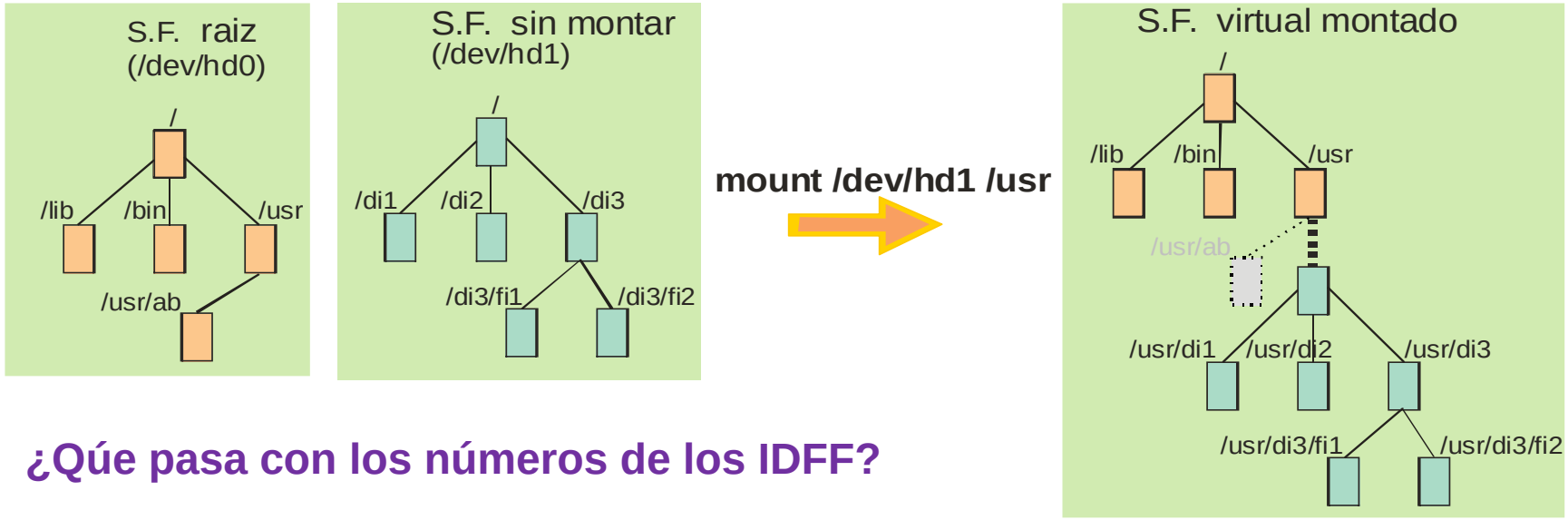
Descripción:

- **Elimina la entrada de directorio y decrementa el número de enlaces del fichero correspondiente**
- **Cuando el número de enlaces es igual a cero:**
  - **Si ningún proceso lo mantiene abierto, se libera el espacio y nodo_i ocupado por el fichero**
  - **Si algún proceso lo mantiene abierto, se conserva el fichero hasta que lo cierren todos**
- **Si se hace sobre un enlace simbólico se decrementa el nº de enlaces del nodo_i simbólico. Cuando llega a 0 se libera dicho nodo_i. No se hace nada sobre el fichero enlazado**
- **Permiso de escritura en directorio de donde se elimina `name`**

MONTADO DE VOLÚMENES O PARTICIONES

datsi © UPM 2022

- **Añade a un árbol de directorios los directorios de un dispositivo. Creando un sistema de ficheros virtual conjunto**
- **El montado es una operación realizada en memoria. No se modifica nada en los discos**
- **Oculta el nombre del dispositivo físico o partición que se monta**



¿Qúe pasa con los números de los IDFF?

- **Si en el directorio de montaje existen ficheros o directorios éstos quedan inaccesibles. (el directorio `/usr/ab` ya no está disponible). Una vez desmontado vuelven a ser accesibles.**
- **En UNIX `/etc/fstab` contiene los sistemas de ficheros disponibles (montados por init) con las opciones de montado para el mandato `mount`**

MOUNT. MONTA UN SISTEMA DE FICHEROS

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int mount(const char *source, const char *target,
 const char *filesystemtype,
unsigned long mountflags, const void *data);
```

Argumentos:

- `source` *:* **sistema de ficheros que se monta, generalmente un dispositivo como `/dev/cdrom`, `/dev/hdb0`, `/dev/sda1` o `/home/imagen.iso`**
- `target` *:* **directorio sobre el que se monta**
- `filesystemtype` *:* **tipo de sistema de ficheros, como "minix", "ext2", "ext3", "ext4", "msdos", "vfat","proc" o "nfs"**
- `mountflags` *:* **Opciones como:**
  - **MS_NOEXEC enmascara los bits de ejecución del sistema montado**
  - **MS_NOSUID enmascara los bits SUID y SGID del sistema montado**
  - **MS_RDONLY montado para lectura solamente**
- `data` *:* **opciones que dependen del tipo de sistema de ficheros.**

Devuelve:

- **Cero o -1 si error**

Descripción:

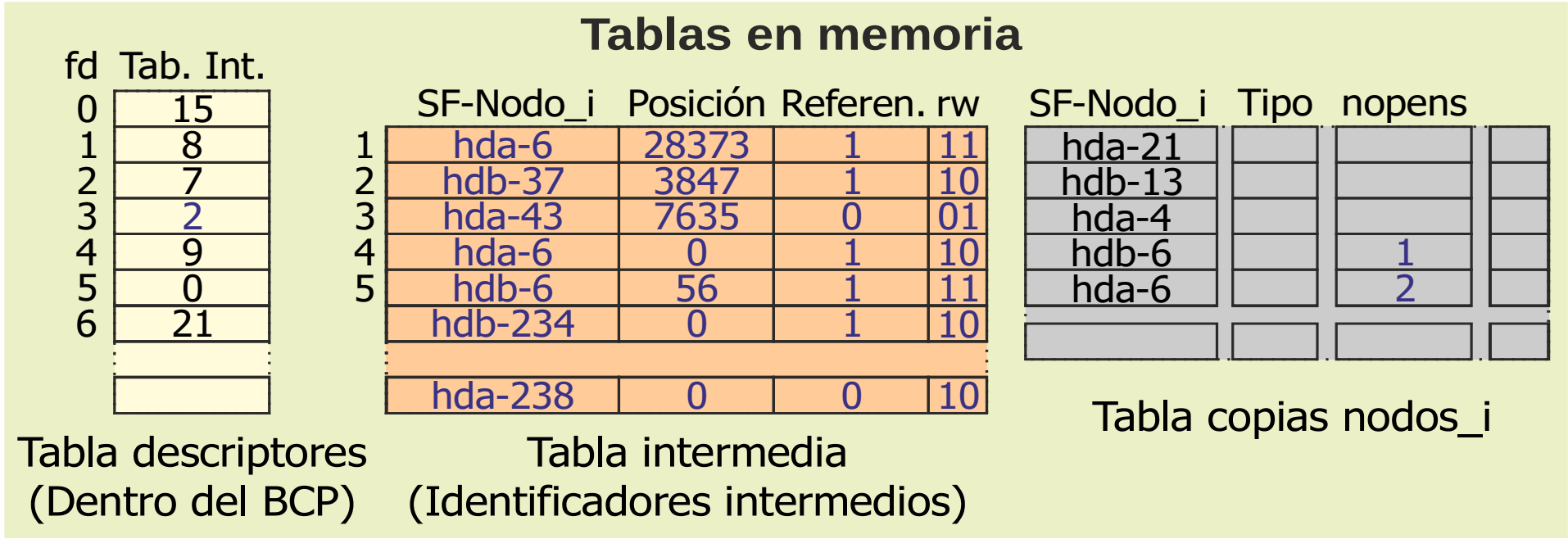
- **Monta un sistema de ficheros**
- **Requiere privilegios de superusuario**

datsi © UPM 2022

Cada sistema de ficheros numera los nodos_i empezando en el 2

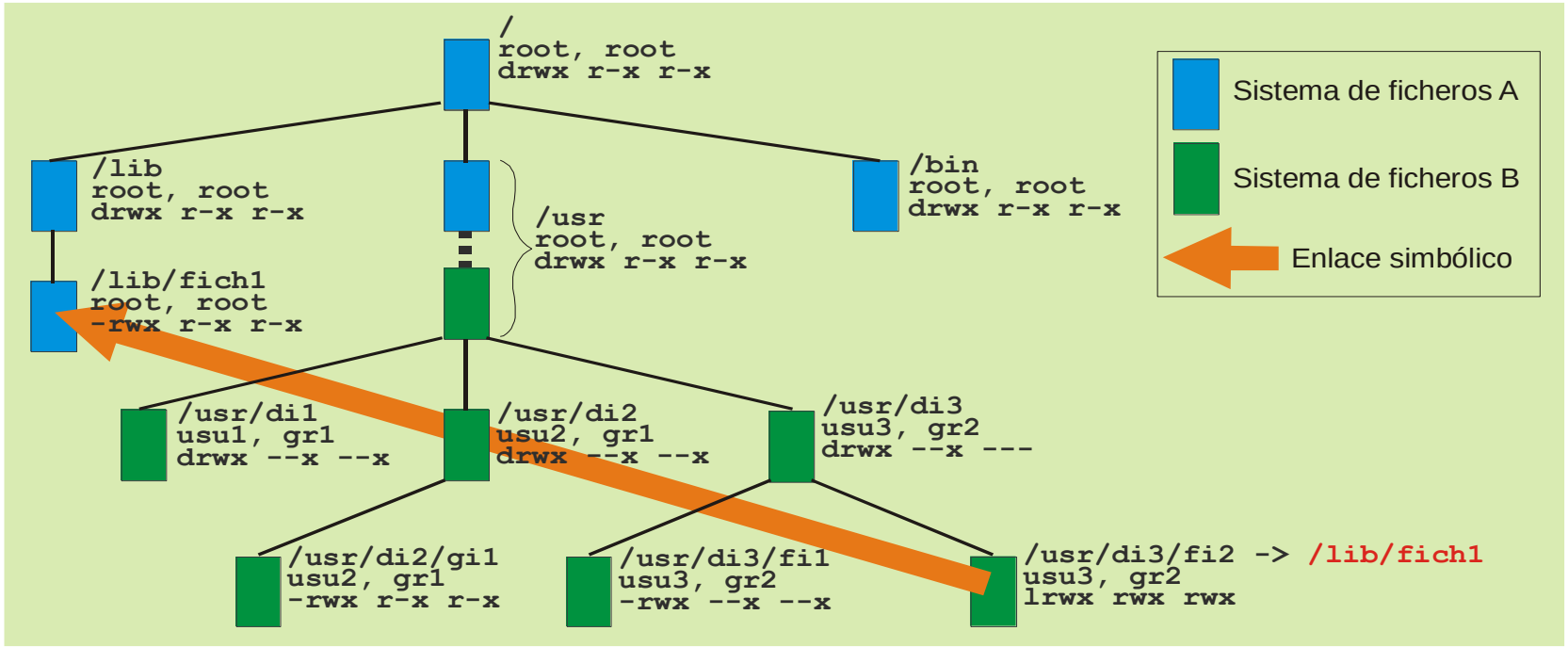
Al montar sistemas de ficheros, los nº de nodo_i no son únicos

Para identificar un nodo_i es necesario especificar el sistema de ficheros y en nº de nodo_i dentro de él. Esto es necesario, por ejemplo, en la tabla intermedia y en la tabla de copias de nodos_i



UNIX: PERMISOS EN VOLÚMENES MONTADOS

datsi © UPM 2022




```
$ ls -l "/usr/di3/fi2"
lrwxr-xr-x root root  10 Mar 11 2004 /usr/di3/fi2 -> /lib/fich1
```

Para abrir /usr/di3/fi2 se comprueban los permisos de

"/" (raíz SF A)
"usr/" (raíz SF B)
"di3/" (SF B)
"/" (raíz SF A),
"lib/" (SF A)
"fich1" (SF A)

(no se comprueban los permisos del nodo_i de "fi2" ni del nodo_i original de "usr/")

UMOUNT. DESMONTA UN SISTEMA DE FICHEROS

datsi © UPM 2022

Servicio:

```
int umount(const char *target);
```

Argumentos:

- `target`: **directorio que se desmonta**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Desmonta un sistema de ficheros**
- **Si hay ficheros abiertos no desmonta y devuelve error**
- **Requiere privilegios de superusuario**



POLITÉCNICA

SERVICIOS UNIX PARA PROTECCIÓN

ACCESS. ACCESIBILIDAD SOBRE UN FICHERO

Servicio:

```
int access(char *name, int amode);
```

Argumentos:

- ***name:*** **nombre del fichero o directorio**
- ***amode:*** **modo de acceso que se quiere comprobar.** `amode` **es el** `OR` **inclusivo de** `R_OK`, `W_OK`, `X_OK` **o** `F_OK` **(comprobar existencia)**

Devuelve:

- **0 si el proceso tiene acceso al fichero (para lectura, escritura o ejecución) o -1 en caso contrario**

Descripción:

- **Utiliza el UID real y el GID real (en lugar de los efectivos) para comprobar los derechos de acceso sobre un fichero**

Ejemplos:

- `access("fichero", F_OK);` **devuelve 0 si el fichero existe o -1 si no existe**
- `access("fichero",R_OK|W_OK);` **devuelve 0 si el proceso que ejecuta la llamada tiene permisos de acceso de lectura y escritura sobre el fichero (utilizando el UID real y el GID real)**

ACCESS. Ejemplo

© UPM 2022

Ejemplo de uso `access` para un programa que tiene activo el bit de SETUID o el de SETGID.

Antes de abrir un fichero se comprueba si el usuario real tiene derechos de acceso. Solamente se abrirá en dicho caso.

```
if (access("archivo", R_OK|W_OK) == 0) {
   /* Se realiza la apertura del fichero */
   fd7 = open ("archivo", O_RDWR);
   .......
} else {
   /* El acceso es negativo, por lo que no hace el open */
   .......
}
```

UMASK. ASIGNA LA MASCARA DE CREACIÓN

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
mode_t umask(mode_t cmask);
```

Argumentos:

- ***cmask:*** **bits de permiso a desasignar en la creación de ficheros**

Devuelve:

- **Devuelve la máscara previa**

Descripción:

- **Asigna la máscara de creación de ficheros y directorios del proceso que la invoca**
- **Los bits activos en la máscara son desactivados en la palabra de protección del fichero (permisos =** `mode & ~umask`**)**
  - **Si máscara = 022 , y se crea un fichero con bits 0777, los bits reales del fichero serán 0755 (resta de los permisos en octal !)**
- **La máscara está almacenada en el BCP del proceso**

CHMOD. CAMBIA LOS BITS DE PERMISO

datsi © UPM 2022

POLITÉCNICA

Servicio:

```
int chmod(char *name, mode_t mode);
```

Argumentos:

- ***name:*** **nombre del fichero objetivo**
- ***mode:*** **Nuevos bits de protección**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **También puede modificar los bits de permiso y los bits SETUID y SETGID del fichero**
  - **S_ISUID = 4000**
  - **S_ISGID = 2000**
- **Sólo el propietario del fichero o el administrador pueden cambiar estos bits**
- **NO se utiliza la máscara**

CHOWN. CAMBIA EL PROPIETARIO DE UN FICHERO

datsi © UPM 2022

Servicio:

```
int chown(char *name, uid_t owner, gid_t group);
```

Argumentos:

- ***name:*** **nombre del fichero**
- ***owner:*** **nuevo propietario del fichero**
- ***group:*** **nuevo identificador de grupo del fichero**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Modifica el identificador del propietario del usuario y del grupo del fichero**
- **Los bits SETUID y SETGID son borrados**
- **Sólo el administrador pueden cambiar estos atributos, salvo el propietario que puede modificar el grupo, si pertenece a más de uno**



SERVICIOS UNIX SOBRE ATRIBUTOS

STAT, FSTAT. INFORMACIÓN SOBRE UN FICHERO

datsi © UPM 2022

Servicios:

```
int stat(char *name, struct stat *buf);
int fstat(int fd, struct stat *buf);
```

Argumentos:

- ***name:*** **nombre del fichero**
- ***fd:*** **descriptor de fichero**
- ***buf:*** **puntero a un objeto de tipo `struct stat` donde se almacenará la información del fichero**

Devuelve:

- **Cero o -1 si error**

Descripción:

- **Obtiene información sobre un fichero y la almacena en una estructura de tipo `struct stat`**

STAT, FSTAT. INFORMACIÓN SOBRE UN FICHERO

datsi © UPM 2022

POLITÉCNICA

```
struct stat {
   mode_t     st_mode;      /* tipo de fichero + permisos */
   ino_t      st_ino;       /* número del fichero  */
   dev_t      st_dev;       /* dispositivo */
   nlink_t    st_nlink;     /* número de enlaces */
   uid_t      st_uid;       /* UID del propietario */
   gid_t      st_gid;       /* GID del propietario */
   off_t      st_size;      /* número de bytes */
   blksize_t  st_blksize;   /* tamaño bloque para I/O */
   blkcnt_t   st_blocks;    /* n° de agrupaciones asignadas */
   time_t     st_atime;     /* último acceso */
   time_t     st_mtime;     /* última modificacion de datos*/
   time_t     st_ctime;     /* última modificacion del nodo_i*/
};
```

Comprobación del tipo de fichero aplicado a `st_mode`:

`S_ISDIR(s.st_mode)`	Cierto si directorio
`S_ISREG(s.st_mode)`	Cierto si fichero normal
`S_ISLNK(s.st_mode)`	Cierto si enlace simbólico
`S_ISCHR(s.st_mode)`	Cierto si especial de caracteres
`S_ISBLK(s.st_mode)`	Cierto si especial de bloques
`S_ISFIFO(s.st_mode)`	Cierto si pipe o FIFO
`S_ISSOCK(s.st_mode)`	Cierto si socket

STAT, FSTAT. INFORMACIÓN SOBRE UN FICHERO

datsi © UPM 2022

POLITÉCNICA

```
int main(int argc, char *argv[])
{
 struct stat infoFich;
 stat(argv[1], &infoFich)
 printf("Número de i-nodo: %ld\n", (long) infoFich.st_ino);
 printf("Número de enlaces: %ld\n", (long) infoFich.st_nlink);
 printf("Propietario: UID=%ld GID=%ld\n", (long)
       infoFich.st_uid,(long)infoFich.st_gid);
 printf("Tamaño del fichero: %lld bytes\n",(long long) infoFich.st_size);
 printf("Número de bloques: %lld\n", (long long) infoFich.st_blocks);
 printf(" Última modificación i-nodo : %s", ctime(&sb.st_ctime));
 printf("Último acceso: %s", ctime(&infoFich.st_atime));
 printf("Última modificación de datos: %s", ctime(&infoFich.st_mtime));
 exit(0);
}
```



SERVIDOR DE FICHEROS
ASPECTOS DE DISEÑO

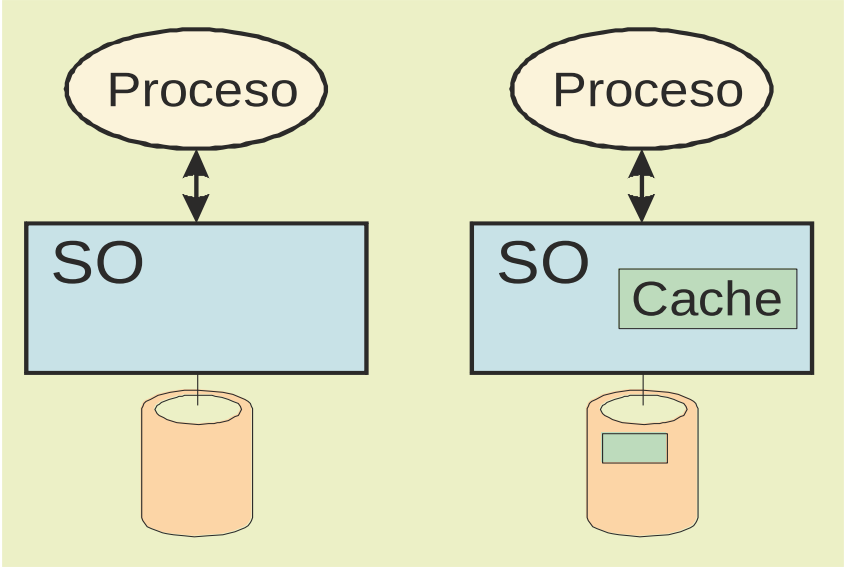
datsi © UPM 2022

POLITÉCNICA

- **Mapas de bits, o vectores de bits: un bit por recurso existente (descriptor de fichero, agrupación). Si el recurso está libre, el valor del bit asociado al mismo es 1, si está ocupado es 0.**
  - **Ejemplo, sea un disco en el que las agrupaciones 2, 3, 4, 8, 9 y 10 están ocupadas y el resto libres, y en el que los descriptores de fichero 2, 3 y 4 están ocupados. Sus mapas de bits de serían:**
    - `MB de agrupaciones:  1100011100011....`
    - `MB de descriptores:  1100011...`
  - **Fácil de implementar y sencillo de usar. Eficiente si el dispositivo no está muy lleno o muy fragmentado.**
- **Listas de recursos libres: mantener enlazados en una lista todos los recursos disponibles (agrupaciones o descriptores de ficheros) manteniendo un apuntador al primer elemento de la lista.**
  - **Este método no es eficiente, excepto para dispositivos muy llenos y fragmentados**
  - **La lista puede construirse dentro de las propias agrupaciones libres.**

CACHE DE BLOQUES

- Estructura de datos en memoria con los bloques más frecuentemente utilizados
  - Lecturas adelantadas
  - Limpieza de la cache (sync)
- Mejora las prestaciones
- El tiempo medio de lectura puede ser distinto del de escritura
- Principal problema: fiabilidad del sistema de ficheros por escritura retardada – cada 30seg por ejemplo- (delayed-write) o diferida (write-back).
  Solución: técnica de write-through, pero no es eficiente.
- Asociado a la gestión de memoria virtual (bloque cache = página)



TUBERÍAS

TUBERÍA O PIPE

Servicio:

```
int pipe(int fildes[2]);
```

Argumentos:

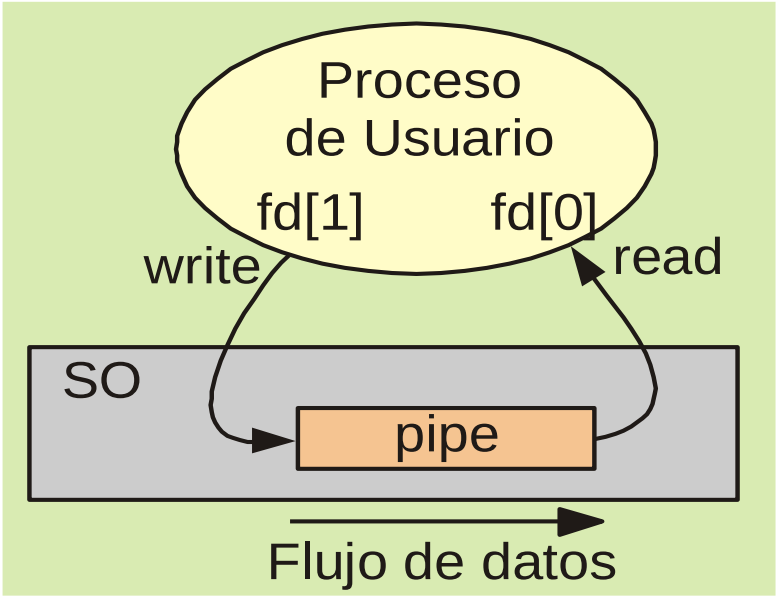
- ***fildes:*** **Vector de dos descriptores de fichero**
  - **fildes[0]: descriptor de lectura**
  - **fildes[1]: descriptor de escritura**

Devuelve:

- **0 o -1 si error**

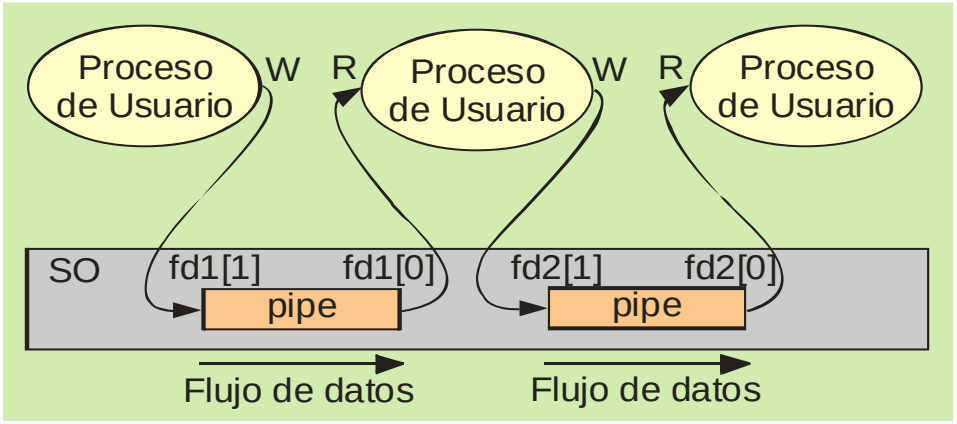
Ejemplo utilización:

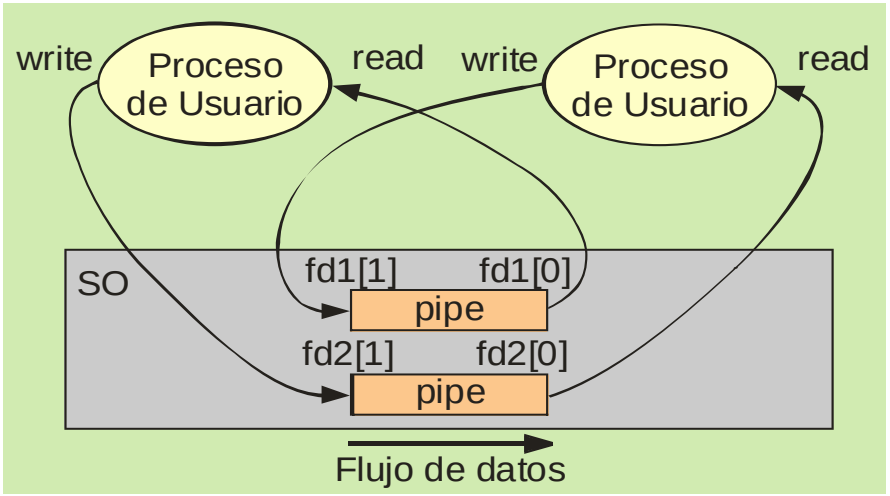
```
int fd[2], ret;
ret = pipe(fd);
```



TUBERÍA O PIPE

- Mecanismo de comunicación y sincronización sin nombre
- Identificación: dos descriptores de fichero
- Sólo pueden utilizarse entre procesos que crean y heredan los descriptores y dentro de la misma máquina
- En cada proceso deben cerrarse los extremos no utilizados
- Con buffering (Tamaño típico: 4 KB)





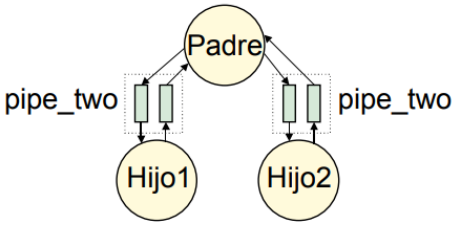


Figura 1.4

SEMÁNTICA DEL READ EN UN PIPE

datsi © UPM 2022

POLITÉCNICA

```
ssize_t read(int fd[0], void *buf, size_t n_bytes);
```

- Pipe vacío → se bloquea el lector
- Pipe con *p* bytes →
  - Si p ≥ n devuelve n
  - Si p < n devuelve p
- Si pipe vacío y no hay escritores (no hay usuarios utilizando `fd[1]`) devuelve 0
- Lectura atómica (cuidado con tamaños grandes, debido al buffering)

!No dejar abiertos los descriptores NO usados!

SEMÁNTICA DEL WRITE EN UN PIPE

datsi © UPM 2022

POLITÉCNICA

```
ssize_t write(int fd[1], void *buf, size_t n_bytes);
```

- Pipe lleno → se bloquea el escritor
- Si no hay lectores (no hay usuarios utilizando `fd[0]`) se recibe la señal SIGPIPE
- Escritura atómica (cuidado con tamaños grandes, debido al buffering)

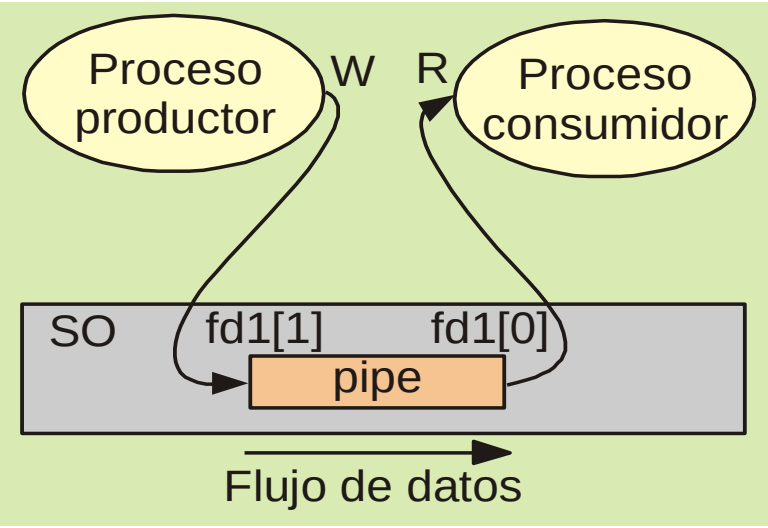
!No dejar abiertos los descriptores NO usados!

ESQUELETO DE EJEMPLO CON PIPES

datsi © UPM 2022

POLITÉCNICA

```
int main(void){
  int fd[2];        /* descriptores del pipe */
  int dato_p[4];    /* datos a producir */
  int dato_c;       /* dato a consumir */
  pipe(fd);
  if (fork() == 0) { /* productor */
    close (fd[0]);
    for(;;){
      /* producir dato_p */
      ......
      write(fd[1], dato_p, 4*sizeof(int));
    }
  } else { /* consumidor */
    close (fd[1]);
    for(;;) {
      read(fd[0], &dato_c, sizeof(int));
      /* consumir dato */
      ......
    }
  }
  return 0;
}
```



TUBERÍA O PIPE – redirecciones de entrada y salida estándar

datsi © UPM 2022

```
int fd[2], ret;

ret = pipe(fd);

pid = fork();

if (pid > 0) // padre

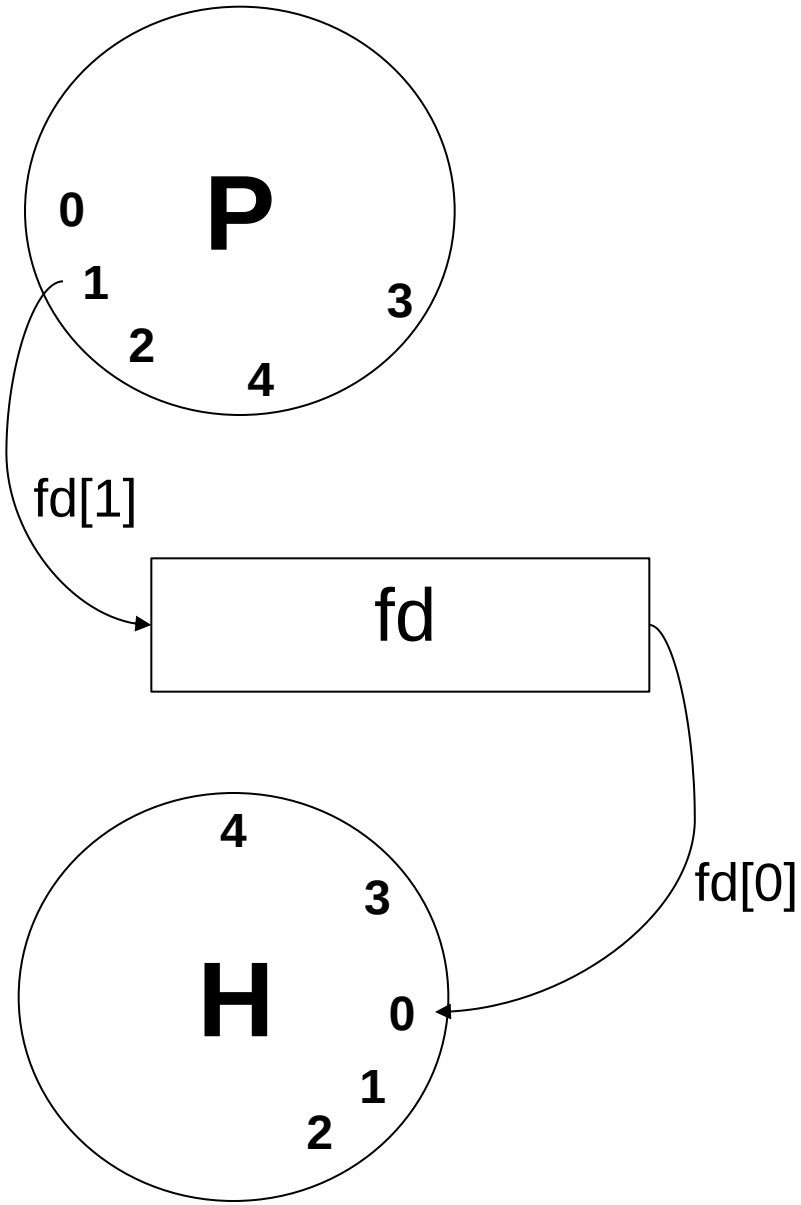
{
  close(1);
  dup(fd[1]);
  close(fd[0]);
  close(fd[1]);

}

else // hijo

{
  close(0);
  dup(fd[0]);
  close(fd[0]);
  close(fd[1]);

}
```



Considere el contenido en un sistema de ficheros de un equipo Unix mostrado en la figura 2.21.

Exponer, en todas las preguntas, los pasos a seguir por el servidor de ficheros para determinar la respuesta dada, indicando el número de línea (de 1 a 18) y el grupo de permisos que interviene.

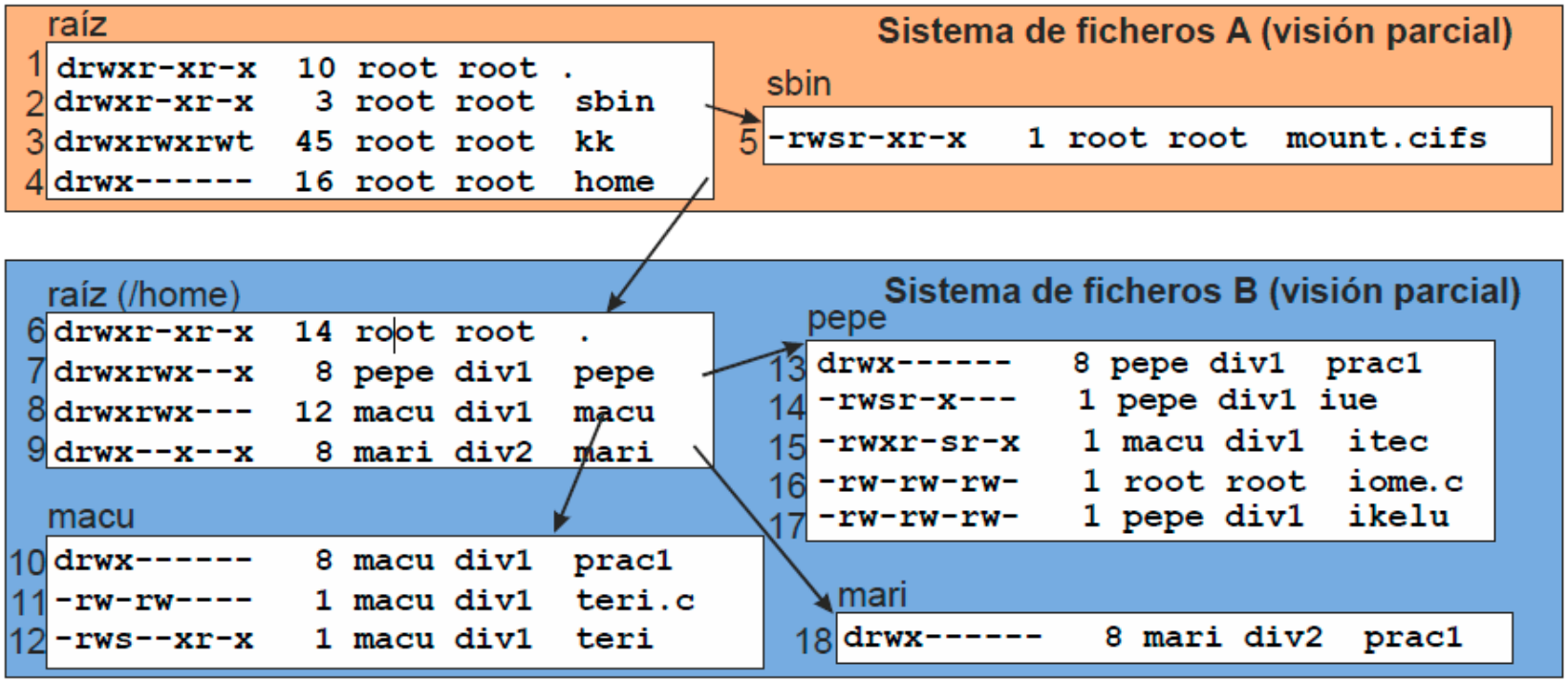
a) ¿Pueden los usuarios pepe, macu *y* mari *acceder a sus directorios?*

b) ¿Puede mari *ejecutar el siguiente mandato?*

```
/home/pepe/itec   /home/macu/teri.c   /home/macu/teri.bak
```

Indicar las identidades reales y efectivas del proceso itec.

c) ¿Podrá itec *abrir para leer y escribir el fichero dado como primer argumento (*teri.c*)? ¿Podrá* itec *crear el fichero dado como segundo argumento (*teri.bak*)?*



User	Group	Mask
root	root	000
pepe	div1	022
macu	div1	027
mari	div2	077

```c
int main(int argc, char *argv[]) {
   int fd, fd2, fd3;
   pid_t pid1, pid2;
   char buffer[1024]="123456789012345678901234567890";

   fd=open(argv[1],O_RDWR);
   pid1=fork();
   if (pid1==0) {
      fd2 = creat(argv[2], 0640);
      fd3 = dup(fd);
      read(fd3,buffer,10);
      read(fd,buffer,10);
      write(fd2,buffer,20);
      close(fd);
      close(fd2);
      close(fd3);
      return 0;
   }
   else {
      pid2=fork();
      if (pid2==0) {
         fd2=open(argv[1],O_RDONLY);
         fd3=open(argv[2],O_RDWR);
         write(fd3, buffer, 10);
         read(fd,buffer,10);
         write(fd,buffer+10,10);
         close(fd);
         close(fd2);
         close(fd3);
         return 0;
      }
      else {
         fd2=creat(argv[2],0755);
         fd3=dup(fd2);
         read(fd,buffer,10);
         write(fd2,buffer,10);
         write(fd3,buffer,10);
         close(fd);
         close(fd2);
         close(fd3);
         return 0;
      }
   }
}
```

Si se ejecuta este programa con los argumentos fich1 fich2 y el fichero fich1 tiene el contenido "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ", responder razonadamente a las siguientes cuestiones:

1. Si el orden en el que se ejecutan los procesos de forma completa es: proceso padre, primer proceso hijo, segundo proceso hijo, ¿cuál es el contenido del fichero fich2 al final de la ejecución?

2. Realizar el mismo ejercicio para el caso de que el orden de ejecución sea el inverso (segundo proceso hijo, primer proceso hijo, proceso padre).

Problema 2.24 (oct-2010)

Sea un sistema de ficheros de tipo UNIX con bloques de 1 KiB, cuya estructura de directorios se indica, parcialmente, a continuación. Asuma que el directorio tmp está vacío inicialmente.

```
tmp  drwxrwxrwt root root
bin  drwxr-xr-x root root
etc  drwxr-xr-x root root
home drwxr-xr-x root root
     rodri drwxr-xr-x rodri prof
     perez drwxr-xr-x perez adm
     lopez drwxr-xr-x lopez adm
     ruiz  drwxr-xr-x ruiz prof
           fich1 -rwxrwxr-- ruiz prof
```

Tres procesos P1 (usuario `rodri`, grupo `prof`) P2 (usuario `ruiz`, grupo `prof`) y P3 (usuario `lopez`, grupo `adm`) ejecutan las sentencias de la tabla adjunta, en el orden de dicha tabla.

Orden	Proceso	
1	P1	`umask(007);`
2	P1	`fd1 = open("/home/ruiz/fich1", O_WRONLY|O_CREAT|O_TRUNC, 0640);`
3	P1	`fd2 = open("/tmp/pru", O_WRONLY|O_CREAT|O_TRUNC, 0660);`
4	P1	`buf = "ABCDEFGHIJK"`
5	P1	`write(fd2, buf, 10);`
6	P1	`lseek(fd2,-8,SEEK_END);`
7	P2	`umask(000);`
8	P2	`fd1 = open("/tmp/pru", O_RDWR, 0666);`
9	P2	`i = 24;  //i está definido como un entero y ocupa 4 bytes`
10	P2	`lseek(fd1,2,SEEK_CUR);`
11	P2	`write(fd1, &i, sizeof(int));`
12	P1	`fork();  //Genera el proceso P4`
13	P4	`lseek(fd2,5200,SEEK_SET);`
14	P2	`lseek(fd1,1024,SEEK_CUR);`
15	P1	`write(fd2, buf, 10);`
16	P2	`write(fd1, &i, sizeof(int));`
17	P3	`fd1 = open("/home/ruiz/fich1", O_RDWR, 0666);`
18	P1	`write(fd1, buf, 10);`
19	P3	`write(fd1, "BB", 2);`